CTVS-2

Asynchronous Requests, Settlement, and Claims

Independent requests, purpose-bound settlement, and immutable full-value recovery.

Technical whitepaper / 29 September 2026

Prepared for Steer Protocol and ecosystem review.

Companion: CTVS-1.

Publication and evidence status

The reference encoding is CTVS-WIRE-2, wire integer 2, Direct Custody Profile (wire profile ID 0).

A proposed standard, not an adopted CIP or frozen deployed interface.

BASELINE REPOSITORY COMMIT

`f5f782de9ba6a7bf6cbb8213995a31734888559c`

Abstract

A vault operation is asynchronous when the user's commitment, the economic exchange, and delivery cannot or should not occur in one transaction. Cardano also permits independent request UTxOs to be submitted without spending a shared accounting state, allowing a later transaction to settle a selected batch. These two motivations - economic delay and submission concurrency - are related but not identical.

CTVS-2 defines common full-fill request ownership and settlement obligations over the CTVS economic boundary. In the specified refundable-escrow semantics, a pending deposit remains nonparticipating until settlement; a pending redemption retains participating shares until economic extinction. The unchanged reference profile creates a fully funded claim. A separately specified native delivery-at-settlement variant pays the final receiver atomically instead. Both admit or extinguish shares once at settlement; later funded-claim delivery never reprices or changes supply. All authorization, accounting, user-bound and output obligations require real on-chain enforcement; present evidence is local and partial.

Settlement may be permissioned. The standard does not require an incumbent protocol to open its scooper or batcher set. It instead separates operator eligibility from custody safety, valuation trust, cancellation rights, and delivery availability. The conservative profile permits refundable pending escrow, one pre-settlement pricing snapshot per batch, fully funded claim outputs, and no partial fills or pre-pricing downstream deployment.

This paper formalizes the lifecycle, batch equations, controller authorization, deadline behavior, request and claim schemas, and infrastructure reconstruction. Worked examples reconcile underlying assets, native shares, and ADA across request creation, mixed-batch settlement, cancellation, and delivery. Direct delivery has an explicit bounded semantic design outside the Direct Custody Profile (wire profile ID `0`). Managed accounting, inventory, epochs and in-flight behavior require their own reviewed implementations; optional names do not imply readiness. Speculative algorithms are deferred rather than presented as part of the initial interface.

Exact Request/Recovery/Claim records and purpose-bound redeemers remain in this paper for the Direct Custody Profile; shared primitive encoding is defined once in CTVS-1 Section 10. Native formats use CTVS-1 Section 2.5's authenticated mapping, not a universal new order datum. Full-value recovery, claim independence, exact pricing and complete allocation are retained.

Common guarantees and explicit implementations

CTVS distinguishes **common economic and integration requirements** from the **fully specified reference construction**, defines a bounded route for native-format conformance, and specifies a semantic direct-at-settlement delivery variant. It does not add a universal transcript language, module framework, permission language, routing engine, or hosted adapter runtime.

Component	Specification
Reference wire	CTVS-WIRE-2; magic CTVS, integer 2, Direct Custody Profile (profile ID `0`)
Reference schemas and fixtures	Exact encodings and fixtures in `candidate-wire/`
Native integration	Explicit semantic mapping and evidence required; no native implementation certified here
Direct-at-settlement delivery	Specified semantic variant outside the Direct Custody Profile; no new wire tag or assigned profile integer
Budget fields	Existing storage, budget and exact-fee fields retained; no silent simplification

Profile boundaries. The Direct Custody Profile fixes its Request spending rules, constructors, and terms-commitment bytes. Native mappings and `delivery_at_settlement` have no assigned on-chain profile here. A native implementation or a distinct reference-wire variant must bind its own reviewed encoding and execution rules. A capability label cannot reinterpret an existing datum field or action.

A conformance claim must identify the actual implementation, supported operation, and evidence. Publication alone certifies none of them.

Implementation status. The historical economic and codec results are retained with their original limits in historical validation evidence. The linked Direct Custody Profile reference has compiled Aiken checks, signed local transaction scenarios, and an accepted-chain reader with local tests [10], [11]. Those are implementation results, not proof of node-backed acceptance, deployment, independent interoperability, or a completed external audit. The illustrative response schema from the earlier source bundle is not claimed to match the current reader byte for byte.

CTVS permits native contracts and datums where they satisfy the common guarantees. Its conformance and delivery requirements are proposals, not claims of verified SundaeSwap conformance.

Figure index and reading key

The figures illustrate the economic and wire rules. The integration response contract is stated in this paper. The reference implementation is linked at a fixed repository commit; its local validation is not a deployment, independent audit, or adoption of CTVS as a CIP.

The figures use a shared spacing grid, thin uniform outlines, restrained line icons and soft blue, violet, green and sand cards. Labels and captions identify the applicable profile. Colour supports grouping; it does not establish authority, success, or a new protocol rule. Arrows identify the relationship named in the figure, not the provenance of individual fungible coins. Dashed treatment marks the locally stated alternative or unsupported path.

Eleven figures are vector SVG/PDF illustrations with selectable text; four figures are image-based. The source package provides the artwork used in the build.

Figure	Placement	Primary question
1. Request lifecycle and economic cutover	Section 3.3	When do backing, supply, and ownership change?
2. Independent request and recovery envelope	Section 4.2	What is created without consuming State?
3. Two-way acknowledgment and coverage	Section 6.3	How do State and every Request prove the same batch?
4. Mixed snapshot settlement anatomy	Section 6.4	What does a complete batch consume, compute, and create?
5. Funded claim versus direct delivery	Section 7.4	How do the two settlement dispositions differ?
6. Recovery independent of settlement	Section 8.1	How can unsupported economics remain refundable?
7. Zero net mint with gross obligations	Section 9.10	Why does zero mint not eliminate validation?
8. Accepted in-flight boundary	Section 10.3	Why can deployed assets no longer be called refundable escrow?

The diagrams explain the written requirements and add no validation evidence. The linked reference implementation has its own dated local results [11]; node-backed acceptance, managed integration, independent review, and deployment remain open.

Contents

Abstract 2

Common guarantees and explicit implementations 3

Figure index and reading key 4

1. Motivation and relationship to CTVS-1 6

2. Actors, authority, and the reference profile 7

3. Ownership and the request lifecycle 9

4. Request UTxOs and controller authorization 12

5. Batch pricing and accounting 14

6. Settlement validation and payment allocation 16

7. Settlement delivery: funded claims and a bounded native variant 19

8. Cancellation, deadlines, and pause behavior 22

9. Worked UTxO transactions and datum representations 24

10. Managed NAV, protocol adapters, and in-flight exposure 28

11. Deferred features and bounded compatibility 30

12. Wire schema and infrastructure interface 31

13. Formal properties and proof sketches 34

14. Economics, scalability, and risk disclosure 36

15. Validation and release requirements 37

16. References 39

Appendix A. Scope and delivery checklist 40

1. Motivation and relationship to CTVS-1

1.1 Two distinct reasons for asynchrony

First, a transaction consuming one accounting state cannot coexist with another transaction consuming the same output. Independent request creation can remove that contention from user submission, even when settlement itself is immediate once selected. Second, a vault may need liquidity realization, a valuation checkpoint, or an authorized downstream action before an economic exchange is possible. An order queue solves the first issue; it does not automatically solve the second.

CTVS-2 gives both cases an explicit boundary without standardizing a universal strategy workflow. A wallet can commit bounded funds, inspect their ownership state, see the relevant execution permissions, and determine whether cancellation or claim delivery is possible.

ERC-7540 provides request/claimable/claimed concepts and allows deposit and redemption asynchrony independently. It does not require one particular timing of economic participation for every implementation. CTVS-2 specifies that timing for its reference profile. Its out-reference identity, cancellation rules, and fixed-destination delivery are Cardano-specific choices, not a claim of literal ERC-7540 compatibility. [1]

1.2 Shared kernel, separate execution

Common semantics and the Direct Custody Profile mechanism are distinct. The comparison below describes the unchanged reference transaction path. A native-format mapping may implement the new Section 7.4 direct-at-settlement variant, but cannot label it as the Direct Custody Profile or reinterpret a Direct Custody Profile Request. No universal execution transcript, routing system or permission language is required.

CTVS-1 defines backing A, economic supply S, accrued asset fees F, reserve R, adjusted balances $X = A + V_a$ and $Y = S + V_s$, and action-specific integer quotes. CTVS-2 reuses them. An asynchronous-only direction needs the shared accounting semantics, not an enabled direct user operation.

Concept	CTVS-1 direct execution	CTVS-2 execution
User commitment	Signed accounting transaction	Funded request UTxO
Economic exchange	Direct state transition	Validated settlement state transition
Price	Consumed state	Declared settlement pricing profile
Delivery	Required direct output	Fully funded claim, then independent delivery
Submission contention	Accounting state consumed	Request creation does not consume accounting state
Recovery	Failed action leaves ordinary inputs unspent	Pending cancellation or funded-claim delivery

An indication of “pending” must therefore say whose property is locked and whether it participates in share P&L. It cannot be a generic progress label hiding different rights.

2. Actors, authority, and the reference profile

2.1 Independent roles

Role	Power	Not implied by that power
Funding owner	Authorizes funds used to create a request	Authority over other holders' shares
Controller	Cancels a pending request under its terms	Ability to reprice a funded claim
Receiver	Receives the fixed successful output	Authority to rewrite the request
Settler / batcher	Constructs an eligible settlement transaction	Arbitrary NAV, destination, fee, or supply changes
NAV authority	Provides valuation under an advertised accounting profile	Custody rights or execution membership
Strategy manager	Acts on permitted investments	Authority to move refundable pending funds
Pauser	Suspends specified economic transitions	Cancellation of already funded claims
Delivery submitter	Pays to deliver a funded claim	Right to take its assets or reserve

A role may be held by a key, multisig, script, or organization. The standard defines the proof and the permitted transition, not the operator's corporate or governance form.

2.2 Permissioned settlement is conforming

A deployment MUST disclose whether settlement is open-key or authorized-key-set in the Direct Custody Profile; capability-authorized or governed execution requires a separately specified extension. Permissioned scoopers and batchers are not required to become permissionless. Published SundaeSwap and Minswap designs demonstrate authorized execution and script-owned orders as relevant Cardano patterns. [3], [4]

Authorization is one necessary predicate. A permitted settler still must satisfy every user limit, receiver condition, value equation, and supply rule. Membership does not establish correct valuation. Where the same entity also signs NAV or manages assets, the combined powers must be disclosed.

2.3 Candidate reference profile

Normative scope. The concrete choices below remain exact for the Direct Custody Profile. A native implementation qualifies through the common mapping and operation requirements of CTVS-1 Section 2.5 plus the applicable CTVS-2 lifecycle rules. It may not claim refundable-escrow or funded-claim guarantees it does not enforce.

The CTVS-WIRE-2 Direct Custody Profile asynchronous component, descriptively `ASYNC-FULL-SNAPSHOT-CLAIM`, adds the following choices to the CTVS direct-custody reference kernel:

Axis	Reference decision
Request types	Exact gross-asset deposit; exact-share redemption
Fill behavior	Full only; no partial consumption or remainder request
Pricing	One authenticated pre-settlement state for all included requests
Economic boundary	Shares issue/extinguish at settlement

Axis	Reference decision
Pending cancellation	Controller-authorized at any time; fixed refund after expiry by anyone
Delivery	Separate fully funded claim; anyone may submit its fixed-destination delivery
Settlement membership	Immutable open-signing-key or sorted authorized-key-set rule
Fees	Per-request economic rounding; exact fixed settlement fee within funded budget
Success storage	Carried with unused budget to the declared economic receiver
Composition	One vault settlement with a closed, validated script-input family
In-flight investment	Not permitted before deposit admission or before redemption funding

The profile preserves a separate claim stage. Direct delivery is an optional CTVS extension, not an assumed substitute for ERC-7540's mandatory claim flow. The reference fixed-destination delivery may be submitted by a third party, but cannot redirect property. [1]

2.4 Exact key-set policy and mode binding

The shared Terms profile is integer 0, using the same authenticated Config and ten-field State as CTVS-1. Async entry is mode bit 4 and async exit is bit 8; capability disclosure does not enable a disabled bit. Unknown profile/mode bits are not ignored. Full fill, consumed-state pricing, funded claims, and carry-to-receiver storage are profile rules rather than caller-selectable strings.

`SettlerPolicy = C(0,[])` permits any explicitly signing reward key. `C(1,[keys])` requires a sorted, unique list of 1..16 permitted key hashes. The chosen Batch reward key must be an explicit required signatory and, in KeySet, a member. Ledger witness validation is separate from the script-visible membership test. Zero fees require no reward output; positive aggregate fees require one distinct output to that key's enterprise address with NoDatum, no reference script, ADA only, and at least the exact aggregate fee. Any extra minimum-output top-up is external, not a higher request charge. [6], [8]

Neither the semantic model nor the wire-format checks execute a script-controller capability. Sending to a script destination and creating requests from a protocol's own inputs do not prove later native script authorization. These are separate integration claims.

2.5 Native async conformance and consent

A native mapping must identify the actual request commitment, cancellation authority, terms, receiver/refund destinations, protected value, pricing basis, economic participation, exact settlement fee and final-delivery form. Shared native inputs require an authenticated allocation of each user's property; mapping one aggregate input to several full-value claims is invalid. The Direct Custody Profile retains its stronger, simpler rule of one exact request and one whole-value refund per source output.

A direct-delivery request must commit that outcome, either in immutable request terms or a verified native operation whose signed meaning makes it unambiguous. An executor cannot replace a promised funded claim with a direct payment, or the reverse, based solely on its own preference. Native cancellation and delivery rights must match the claimed CTVS semantics; otherwise expose only the actual read-only position and list the missing execution guarantees.

As a priority acceptance case, use one real script-controlled treasury/portfolio: fund the request under its actual guard, enforce successful return to its committed destination, and permit only its actual supported fixed cancellation. Sending an output to a script does not execute that guard. Key-only Direct Custody Profile remains unchanged; no actual script-controller integration is certified here.

3. Ownership and the request lifecycle

3.1 State machine

```
                          settlement
PendingEscrow --------------------------------> Claimable
     |                                             |
     | controller cancellation                     | fixed delivery
     | or permissionless expiry refund             |
     v                                             v
Refunded                                       Delivered

An off-chain selection notice changes none of these rights.
```

The chain's consumption of the request is the commitment point. The words "selected," "scheduled," or "batcher accepted" are provider observations unless a separately defined on-chain transition has occurred.

Native direct-delivery alternative. Section 7.4 permits `PendingEscrow -> Delivered` through one validated settlement transaction. The ownership cutover and all gross economic obligations are the same; no intermediate `Claimable` record is emitted. The diagram above remains the Direct Custody Profile lifecycle. An unfunded or accepted-in-flight position does not fit either path until its separately specified rights are satisfied.

3.2 Deposit ownership

While pending, the offered assets belong to the request's contractual refund/settlement entitlement. They are not share backing and earn no vault share P&L. Settlement admits the net assets and issues shares into a fully funded claim. Those shares participate immediately, even when unclaimed. Delivery later moves already issued shares to the fixed receiver.

Deposit stage	Backing A	Supply S	User's property
Pending	Unchanged	Unchanged	Bounded, refundable asset escrow
Claimable	Increased by admitted net assets	Increased by issued shares	Fixed share quantity, participating in vault P&L
Delivered	No delivery-related change	No delivery-related change	Those shares at receiver destination
Refunded	Unchanged	Unchanged	Original offered assets and refundable ADA

3.3 Redemption ownership

A pending redemption locks existing shares. It does not burn them or remove them from participating supply. Settlement determines gross asset value, extinguishes the shares, accrues any exit fee, and moves the net asset payout into a funded claim. That asset claim no longer participates in vault-share P&L.

Redemption stage	Backing A	Supply S	User's property
Pending	Unchanged	Unchanged	Locked participating shares
Claimable	Reduced by gross redeemed assets	Reduced by extinguished shares	Fixed underlying-asset claim
Delivered	No delivery-related change	No delivery-related change	Those assets at receiver destination
Refunded	Unchanged	Unchanged	Original shares and refundable ADA

This profile deliberately does not burn at request creation. A different ownership model needs a separate accounting specification and cannot reuse these equations unchanged.

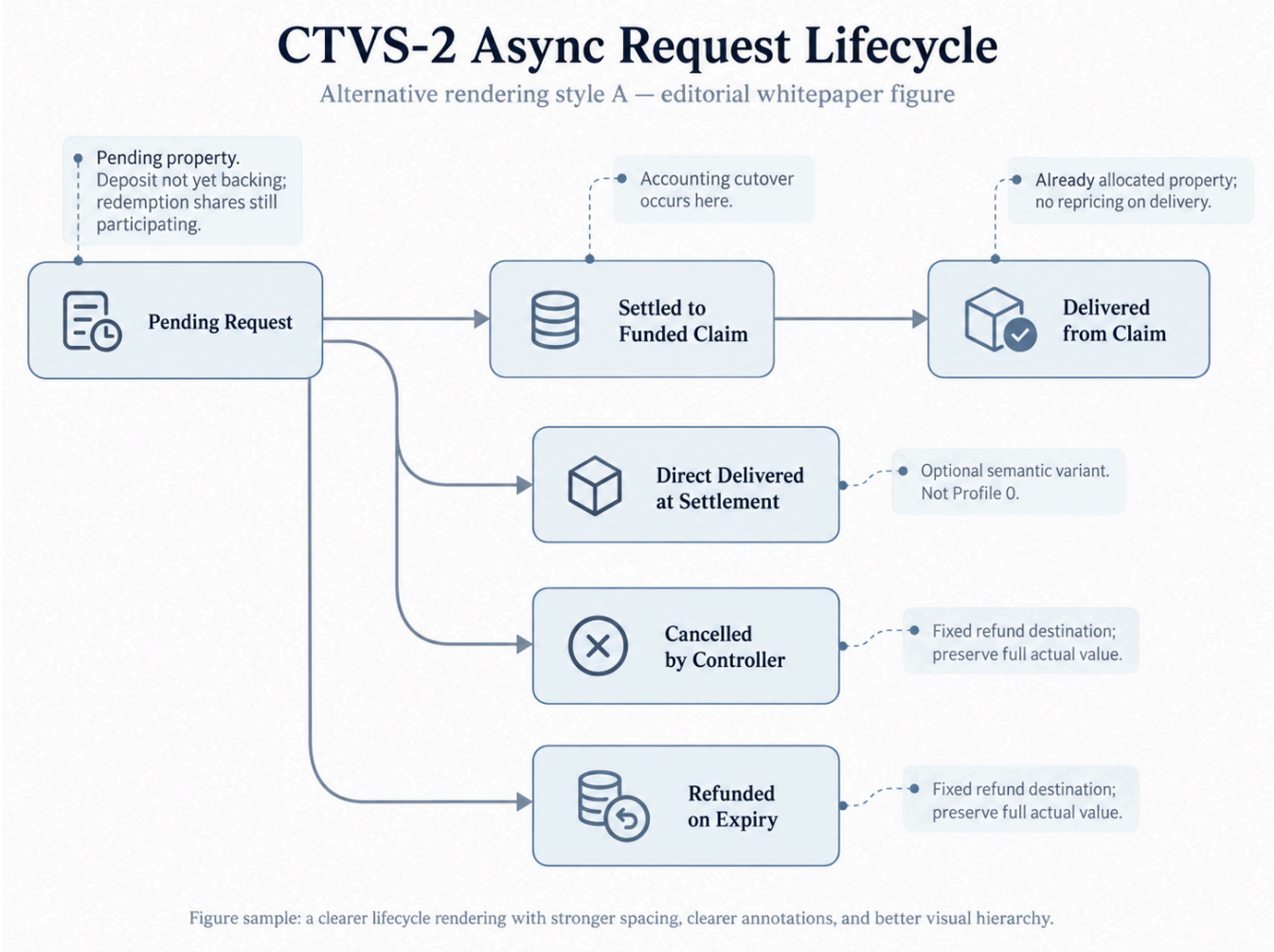


Figure 1: *Request lifecycle and the single economic cutover. Pending requests do not change backing or supply; settlement changes participation exactly once; later claim delivery changes location only. Direct-at-settlement is a separate variant outside the Direct Custody Profile.*

3.4 An economic obligation is not yet claimable

For the direct-at-settlement variant there is no claim object or claimable phase: the final protected output is already delivered. For the funded-claim variant, the requirements below are mandatory. These statuses must not be conflated by an indexer.

A vault may owe an exit while still lacking liquid underlying. That is not a funded claim under this standard. A claim MUST contain the exact promised economic asset and quantity, plus its protected delivery reserve. A position token, a manager's promise, or a future withdrawal right cannot be displayed as an immediately deliverable claim for another asset.

No claim delivery depends on the accounting state, current NAV, or the original batcher. Operational access to the network and funding for a valid delivery transaction are still necessary.

3.5 Full economic histories, not phase-only equivalence

Two histories ending in the same phase labels can have different backing, supply and claims. At $V_a = 1, V_s = 2$, zero fees, a reachable $A = 2, S = 3$ follows from minting three shares for two assets at bootstrap. Consider a one-asset deposit and a three-share redemption in separate settlements:

Order	First result	Second result	Final A / S	Asset claim
Deposit, then redeem	Issue 1 share; A=3,S=4	Redeem 3 for 2 assets	1 / 1	2
Redeem, then deposit	Redeem 3 for 1 asset	Deposit 1 for 1 share	2 / 1	1

Both requests are claimable in both histories. Snapshot pricing is not violated; different consumed states produced different quotes. A visited-state key containing only phases is nevertheless unsound for general economic exploration. The corrected state key includes phases, backing, supply, fees and claim quantities. Its three-request finite fixture reaches 96 economic states over 64 phase combinations and 213 transitions. This is not an unbounded lifecycle or changing-NAV proof. [13]

3.6 Independent objects and non-authoritative queue aggregates

Config/State authenticate the vault economy; Request uses its originating output reference and supported recovery envelope; Claim uses its own out-reference and verified settlement lineage for authentic indexing. A datum is not proof that its named controller originated the request. A valid funded candidate is not admitted backing, an execution reservation, or a global queue registration.

Independent creation cannot maintain an exact pending count in the accounting State. Derive such totals at an identified chain point or introduce a separately authenticated registration protocol. Do not close or sweep the vault merely because local supply is zero. In this reference profile, authenticating a STATE-bearing own input precedes all datum dispatch, preventing custody from being mistaken for refundable request property. [6]

4. Request UTxOs and controller authorization

4.1 Request identity and creation

The initial request ID is its originating output reference, derived after the creation transaction exists. The datum MUST NOT contain its own transaction hash as a self-referential identity requirement. A batch refers to this stable ID, not its current position in an input array.

```
request_id = (origin_transaction_hash, origin_output_index)
```

Creating an output at a script address does not itself run that output's spending validator. Consequently, well-formed-looking requests are not automatically authenticated admissions or capacity reservations. A settler must validate the complete request and value when consuming it. A provider must distinguish a funded candidate from an authenticated statement that the alleged controller initiated it.

An optional creation beacon can validate the creation path and improve discovery, but requires its own minting/lifecycle rules. CIP-89 provides a beacon design pattern; this reference profile does not require a beacon or NFT per request. [5]

4.2 Stable recovery envelope and separately decoded economics

The current candidate has exactly four outer Request fields:

```
Request = C(1,[magic,wire_version,recovery,economic_body])
Recovery = C(0,[vault_policy,controller,refund,deadline_posix_ms])
RequestBody = C(0,[terms_hash,kind,offered,minimum_output,
                   receiver,storage_lovelace,
                   execution_budget,settler_fee])
```

`magic` is bytes CTVS and `wire_version` is integer 2. Recovery contains a key-controller constructor and a complete Destination. The outer `economic_body` is **Data**, not an eagerly validated RequestBody. Settlement interprets the supported eight-field body; cancellation/expiry first validates only the outer and recovery structure. This implements the separation of recovery from settlement eligibility at the datum boundary. Exact recursive primitive rules are in Section 12 and CTVS-1 Section 10. [6], [7]

Unknown economic constructors, zero economic amounts, a bad terms hash, invalid budget or invalid success receiver can prevent settlement without disabling an otherwise valid fixed refund. Do not downcast the complete economic request before branching on Cancel. An unknown outer version, malformed recovery structure, invalid refund destination or unsupported controller has no implied recovery path.

The profile fixes FullOnly, snapshot pricing, claim delivery and CarryToReceiver. No optional epoch/pricing strings exist in this body; those require distinct reviewed encodings. For settlement, require correct vault/terms, enabled mode, supported kind, positive offered quantity and minimum, exact funding, bounded fees and full completion. For recovery, no current state, NAV, pause or batcher is required.

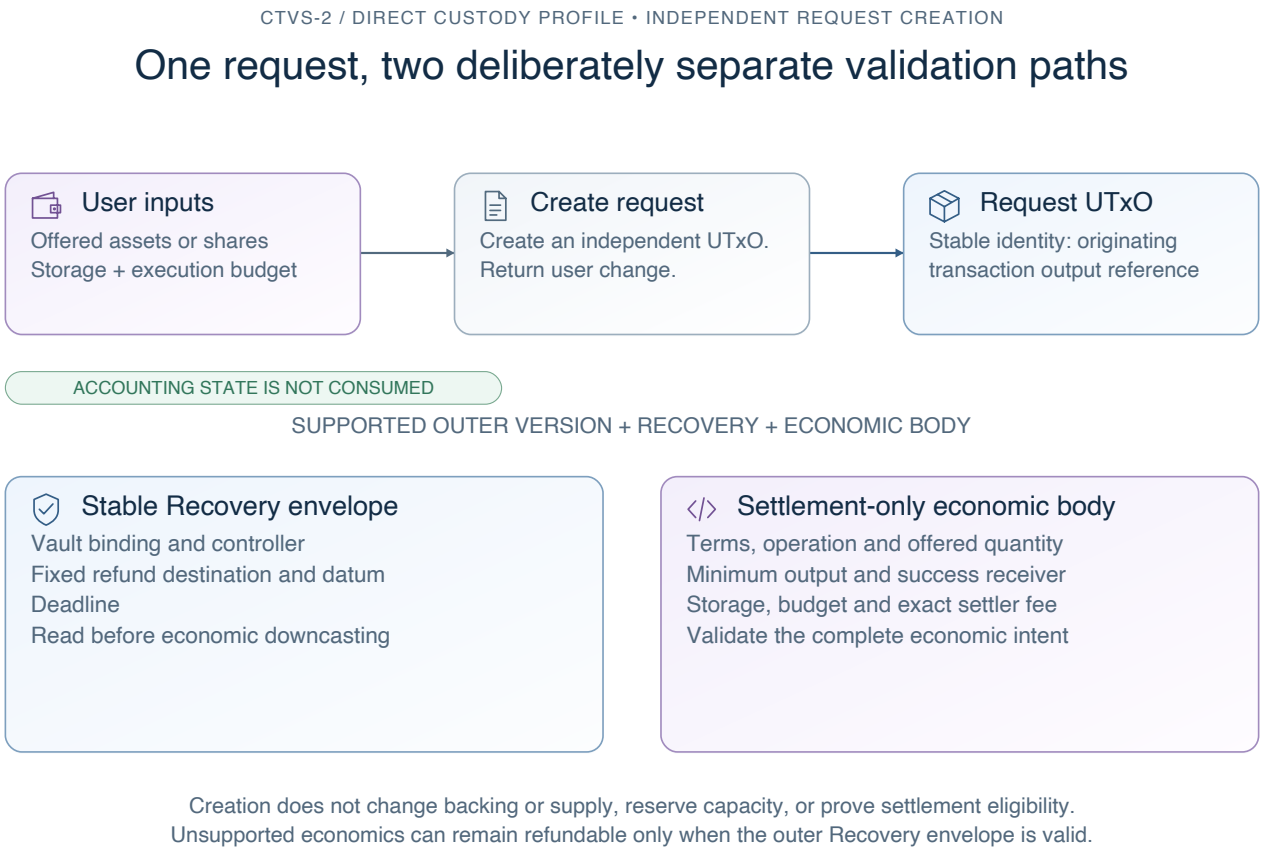


Figure 2: *Independent request creation and the stable recovery envelope. No State is consumed and no shares are issued. Recovery data is decoded separately from the settlement-only economic body so unsupported economics can remain refundable under a recognized envelope.*

4.3 Value partition

Let d be offered deposit assets, r offered redemption shares, R_u protected storage lovelace, B_u funded execution budget, and e_u the exact authorized settler fee with $0 \le e_u \le B_u$.

For a native-token deposit, the request contains exactly d underlying tokens and $R_u + B_u$ lovelace. For an ADA deposit:

$$V_u(\text{ADA}) = d + R_u + B_u. \tag{1}$$

For a redemption, it contains exactly r shares and $R_u + B_u$ lovelace. No other native asset is admitted in this reference request shape. The execution budget is not backing and does not buy shares.

At settlement, the economic claim carries $R_u + B_u - e_u$ lovelace, plus any external top-up. The settlement fee is paid separately. Under the declared `CarryToReceiver` policy, this storage/unused-budget amount follows the successful economic receiver. On cancellation, the entire $R_u + B_u$ follows the refund destination. A different success-refund destination requires an explicitly funded additional-output profile; the same ADA cannot be both carried to a receiver and refunded elsewhere.

Fee/budget decision. The existing three fields R_u, B_u and exact e_u are retained in the Direct Custody Profile. Since execution is full and occurs once, the algebraic reparameterization $r_u = R_u + B_u - e_u$ gives request funding $r_u + e_u$,

successful carry r_u and cancellation return $r_u + e_u$. It can simplify an off-chain display, but this specification does **not** remove or reinterpret the original fields. A future native mapping may use a different internal representation only after proving the same user-authorized allocations. Variable per-execution fees and replenishable lifetime budgets are not added.

4.4 Key controllers and capability extensions

The reference Controller is exactly `C(0,[key_hash])`. Before expiry, controller cancellation requires that key among explicit required signatories and a valid ledger witness. The controller need not be the economic receiver or funding owner; attribution of request creation is separate from its cancellation guard. The Direct Custody Profile does not encode a generic ScriptController.

A future script-controller profile must identify the actual authorization purpose or authenticated capability, action domain, replay rules, permitted value/datum successor, and independent payout allocation. A reference input or unrelated script hash is not authority. A governance capability may legitimately evolve its datum, so exact restoration is a chosen profile rule, not a universal authorization solution. It cannot become an unreviewed exception to the restricted input family. A real protocol-owned-user test is required before claiming that support. [6], [8]

4.5 Own-input role checks and funding boundaries

Resolve each request from its actual spending-purpose out-reference. Inspect protected identity assets before treating a datum as Request: a STATE-bearing input must satisfy the State branch; an invalid STATE quantity or unexpected ID fails. A request-looking datum cannot send real state custody through fixed refund.

No attached reference script is allowed on protected request/claim outputs in this candidate. For settlement, extra native assets invalidate the closed Request value; for supported fixed refund, preserve every actual input asset instead of discarding the surplus. Unknown or oversized economics must not be silently admitted because the recovery envelope decodes. Full recovery still depends on constructing a valid output and transaction. [6]

5. Batch pricing and accounting

Reference algorithm, mandatory economic obligations. Sections 5–6 retain the exact snapshot/full-fill/Direct Custody Profile construction and proofs. A native implementation uses its declared, independently reproducible pricing and supply algorithm under the common contract; it must not claim this snapshot profile when it reprices sequentially. The direct-delivery variant in Section 7.4 below uses the same snapshot economics in this specification; only the output disposition differs. No generic transcript interpreter is introduced.

5.1 One pre-settlement snapshot

Let the consumed state be (A_0, S_0, F_0), with $X = A_0 + V_a$ and $Y = S_0 + V_s$. All included reference requests are priced against this same pair, identified by the exact consumed state reference. The ten-field direct State has no independently mutable checkpoint field. The batcher cannot choose between snapshot and sequential pricing per request.

For deposit i with gross input d_i:

$$f_i = \text{ceil}\left(\frac{d_i b_e}{10{,}000 + b_e}\right), \qquad n_i = d_i - f_i, \qquad q_i = \text{floor}\left(\frac{n_i Y}{X}\right). \tag{2}$$

For redemption j with shares r_j:

$$g_j = \text{floor}\left(\frac{r_j X}{Y}\right), \qquad h_j = \text{ceil}\left(\frac{g_j b_x}{10{,}000 + b_x}\right), \qquad w_j = g_j - h_j. \tag{3}$$

Every request enforces its minimum output and positive economic result. Rates and virtual parameters are authenticated terms. Economic fees are rounded per request, not once over an aggregate that the batcher can split differently.

CTVS-1 Sections 6.5 and 6.6 define common units and optional verified pricing assertions. A native settlement profile using `Price` MUST identify the coherent accounting basis used for each included request and enforce its declared snapshot or sequencing rule. A pending estimate does not fix a future rate. This optional interface does not change the reference snapshot formulas or enable withdrawals in the Direct Custody Profile. Once a Claim is funded, its ledger-unit entitlement remains fixed through delivery.

CTVS-1 Section 6.7 supplies the complete construction and decoding walkthrough and the isolated provider binding. The same envelope is used here; no second async Price encoding is defined. The example's custody checkpoint does not implement request settlement or funded-claim delivery. A native async implementation must additionally bind the verified assertion to the authenticated request, settlement basis, actual share issuance or burn, and funded output. It cannot issue an entitlement based only on a future tend estimate. Signed reports and managed composition remain unsupported by the isolated example.

5.2 Aggregate state transition

Let D and W denote the included deposit and redemption sets. The successor is:

$$A_1 = A_0 + \sum_{i \in D} n_i - \sum_{j \in W} g_j, \tag{4}$$

$$S_1 = S_0 + \sum_{i \in D} q_i - \sum_{j \in W} r_j, \qquad F_1 = F_0 + \sum_{i \in D} f_i + \sum_{j \in W} h_j. \tag{5}$$

The mint/burn profile requires:

$$M(\text{share}) = \sum_{i \in D} q_i - \sum_{j \in W} r_j. \tag{6}$$

At the physical underlying boundary:

$$(A_1 + F_1) - (A_0 + F_0) = \sum_{i \in D} d_i - \sum_{j \in W} w_j. \tag{7}$$

The reserve of the accounting state does not change during settlement. Requests and claims retain their own ADA partitions.

5.3 Feasibility and netting

The batch validates nonnegative bounded successor balances, the final backing cap, aggregate participating share extinction, and full funding of all claims and fee obligations. Redemption requests must hold pre-existing shares; newly issued outputs in this transaction are not pending redemption inputs. The reference batch explicitly requires $\sum r_j \leq S_0$.

Deposit inflows may fund simultaneous redemptions only when all included operations settle atomically at the same authenticated pricing rule. Netting is a physical funding optimization, not permission to issue an unfunded claim. For managed profiles, physical spendable liquidity must be demonstrated; a positive A_1 alone is not enough.

A final-balance cap allows deposits and redemptions to offset in one atomic transaction. A different gross-flow cap is a separate advertised policy. Limits must not be left to a batcher's undocumented interpretation.

5.4 Snapshot equality is not global fairness

Included requests use the same rational reference price, but per-request rounding and fees can produce different effective rates at different sizes. This is disclosed behavior. The batcher may select which eligible requests to include unless a stronger authenticated ordering profile exists.

Do not promise global FIFO or all-request pro-rata fulfillment from local validation of a chosen set. A script generally sees its transaction context, not an exhaustive proof of every older eligible request in the UTxO set. Ordering commitments and availability proofs are additional protocol machinery.

5.5 A zero-net-mint batch

At $A_0 = S_0 = 1{,}000$, $V_a = V_s = 1$, and zero fees, a ten-asset deposit issues ten shares while a ten-share redemption pays ten assets. $A_1 = A_0$, $S_1 = S_0$, and the net share mint is zero.

The deposit's shares can be funded from the redemption input while the redemption's assets are funded from the deposit input. Gross obligations still exist. The state and request validators must prove both, even though the ledger minting policy for the share asset may not execute for a zero-net entry. Treating "no mint" as "no issuance validation needed" is invalid.

5.6 Wire witnesses are not economic authorities

A Batch entry contains only `request_ref`, `claim_output`, and externally funded `claim_topup`. It does not supply an authoritative quote, gross debit, fee, or share-total field. Recompute all economic quantities from the covered RequestBody, authenticated Terms and consumed State. The list is sorted by transaction-ID bytes then output index for deterministic interpretation, not sequential repricing.

Verify the final backing cap, economic and physical aggregate bounds, enabled mode/pause for each direction, existing redemption inputs, positive results and every user's minimum. The maximum-deposit algorithm in CTVS-1 Section 6.4 remains an economic limit calculation; it is not a promise that this transaction fits. [6]

6. Settlement validation and payment allocation

6.1 Accounting transaction and exact role envelopes

```
Reference: immutable authenticated Config
Spend:     State + all selected Request inputs + key funding
Redeemers: StateSpend(Batch), each RequestSpend(Acknowledge),
           MintPurpose(SupplyUpdate) only for nonzero SHARE mint
Produce:   State successor + one Claim per request
           + optional reward + external-funder change
```

StateSpend, RequestSpend, ClaimSpend and MintPurpose are role tags 0, 1, 2 and 3, each wrapping the three fields `magic`, `wire_version`, and `action`. Tags are scoped to their type; a request action zero is not a state action zero. The authentic input role must agree with the envelope. CTVS-1 Section 4 defines the per-vault script and protected-asset dispatch. Section 12 below defines the exact actions.

6.2 Two-way request/state acknowledgment

Each request's `C(0,[state_ref])` acknowledgment requires an ordinary spending input containing the authentic STATE(1) at the expected full vault address. Locate its actual `Spend(state_ref)` redeemer and require StateSpend with inner Batch tag 4. Merely referencing State, naming a state reference, or consuming State for Pause or TopUpReserve does not authorize settlement.

The Batch branch identifies **all ordinary inputs at its vault script except its one authentic State**, before filtering by economic validity. Every such input must be a supported Request whose RequestSpend acknowledgment points back to this exact state. A cancellation redeemer cannot be treated as a settlement acknowledgment. A malicious self-reported Boolean is not evidence that the state validator executes. The ledger must actually run all required handlers.

6.3 Exhaustive request coverage

Let $\mathcal{R}$ be those consumed request references and E the Batch entries. Require:

$$|E| = |\,\mathrm{unique}(\mathrm{refs}(E))|, \qquad \mathrm{set}(\mathrm{refs}(E)) = \mathcal{R}. \tag{8}$$

Entries have deterministic lexicographic out-reference order, no duplicates, and count at most the authenticated Terms.max_batch and structural cap. Every covered candidate passes complete recovery, economic, value, version, terms, mode, expiry and quote checks. Do not drop an invalid consumed input from the list while retaining its value.

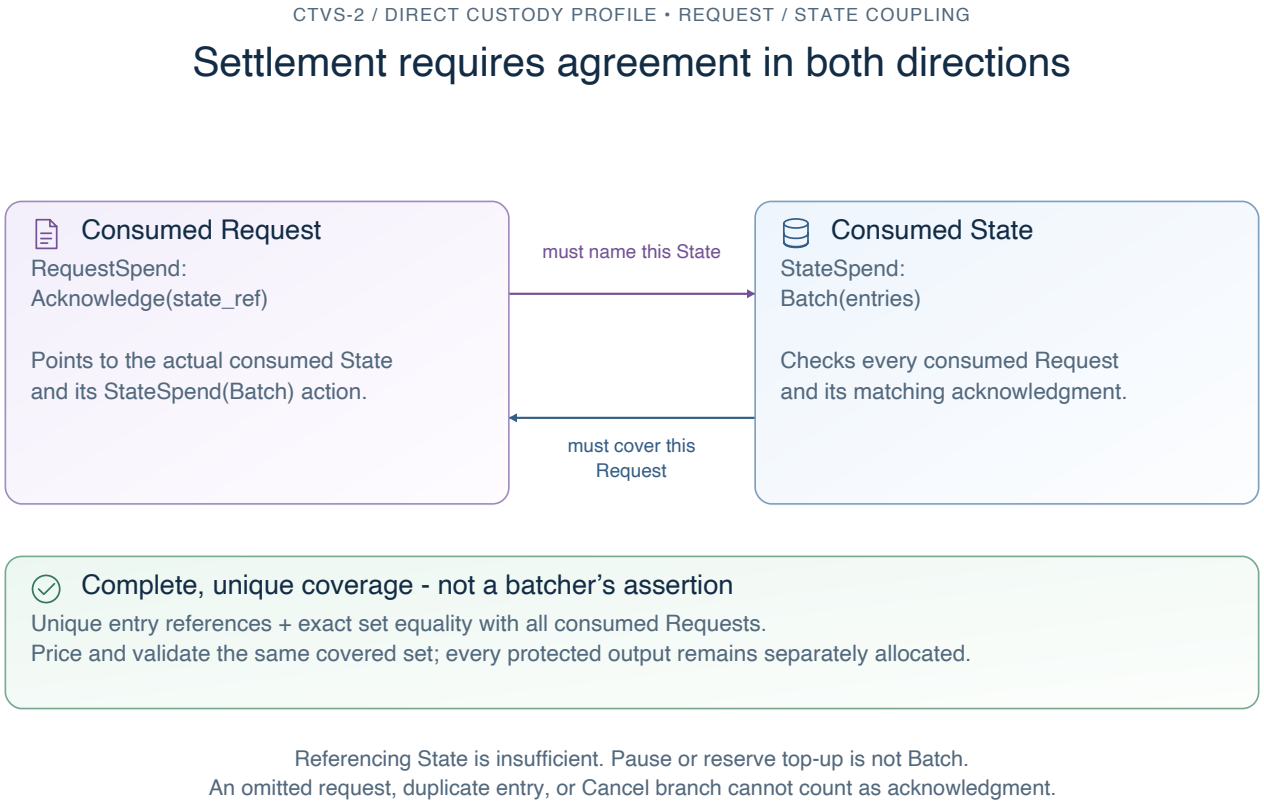


Figure 3: *Two-way acknowledgment and exhaustive coverage. The State Batch entry set equals every consumed Request, and every Request points back to the same authentic State Batch action. A reference input or self-reported flag cannot substitute for these checks.*

6.4 Gross economics, successor and output obligations

Recompute all quotes against the same old State. Check each positive input/output, minimum, exact fee and budget. Verify aggregate backing, fees, supply, existing shares and liquidity, and then the exact closed successor and net SHARE mint. A zero-net-mint batch still checks gross issuance/extinction and all claims. Config and ID/STATE supply are immutable.

Each entry constructs one Claim at the shared Q script binding `request_ref`, `state_ref`, policy, terms, asset, amount, receiver and carried ADA. The successor, every claim, and positive reward have distinct allocated indices within the actual output count. Same receivers do not merge claim obligations. Top-ups are externally funded, disclosed in the entry, and included in Claim.carried_lovelace; they do not buy shares or increase a request fee.

The reward key is explicitly required to sign and satisfies the configured key policy. Zero summed request fees require None reward output. Otherwise the reward output is ADA-only at that key's enterprise address, with NoDatum and no reference script, and at least the exact sum; any extra ADA is external. All protected role/output conditions and accounting partitions remain necessary even when the transaction balances.

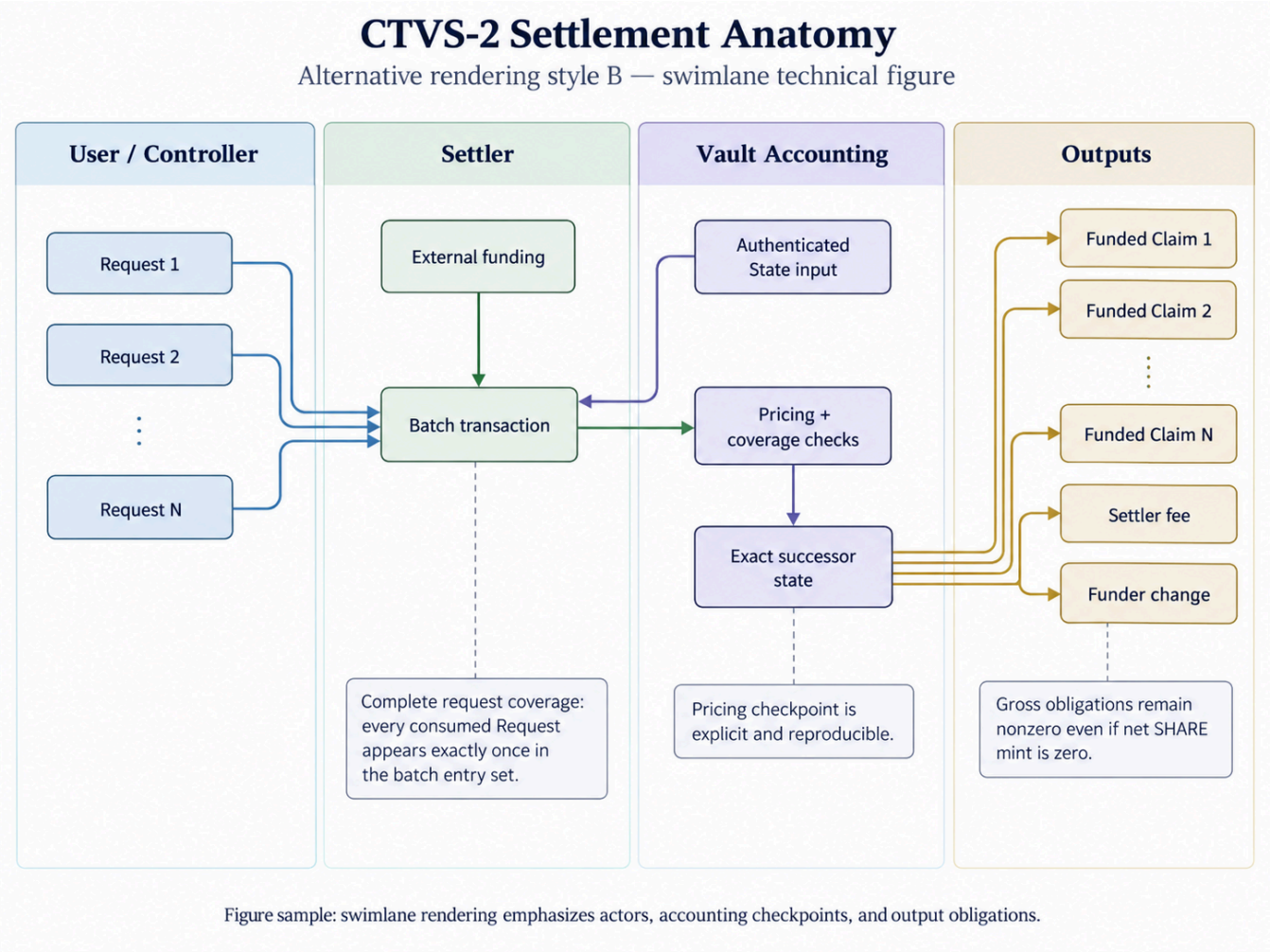


Figure 4: *Mixed snapshot settlement anatomy. All requests price against one old State, the transaction validates gross issue and extinction, and each request receives its own funded result. Physical netting does not remove user obligations.*

6.5 Closed validation domain and receiving scripts

These restrictions remain unchanged for the Direct Custody Profile. Alternative native compositions must prove the complete source and payout obligation domain through their actual validators. Neither a mapping label nor local output injectivity alone permits removal of the restrictions.

The Direct Custody Profile permits one vault State, its Requests, and key funding only. It rejects other script-spending families, concurrent claim delivery, unrelated mint, all withdrawals (including zero), certificates, votes, proposals and treasury fields. Local injectivity alone is not proof against an unrelated validator sharing a payout. Arbitrary cross-vault atomic composition needs a common allocation verifier or reviewed adapter.

Claims have their own datum at Q; the final receiving script retains its own exact datum. Creating a destination output does not execute the recipient script. The no-reference-script and credential/datum rules apply to all protected outputs. These restrictions do not force permissioned protocols to change their operator sets.

6.6 What the executed link tests establish

The local `verify_settlement_links()` function exercises linkage, role, coverage and selected boundary checks. It is **not** a full settlement validator: quote correctness, recipient amounts, successor accounting, net mint and complete value conservation remain separate obligations. The economic model and wire linkage predicates have not been compiled or proven jointly equivalent to a complete Cardano implementation. [6], [13]

7. Settlement delivery: funded claims and a bounded native variant

7.1 Exact claim creation

Direct Custody Profile / funded-claim branch. Sections 7.1–7.3 remain the exact existing rule. Section 7.4 specifies the separate direct-at-settlement semantic variant. No Claim constructor or Delivery redeemer has been changed.

A share claim contains shares already issued at settlement; an asset claim contains the exact net underlying payout already removed from remaining backing. The Claim record has ten fields (Section 12), including consumed request/state references, not a self-referential future transaction ID.

For a request reserve R_u, execution budget B_u, fixed fee e_u and external top-up t:

$$R_{claim} = R_u + B_u - e_u + t. \tag{9}$$

A token claim holds the exact economic token quantity and R_{claim} ADA. An ADA claim holds economic lovelace plus R_{claim}. Legitimate creation has no unexplained surplus and requires positive carried ADA. No NAV report, position token or future liquidity promise substitutes for the asset promised in a funded claim.

7.2 Shared fixed-delivery domain

The independently defined claim script Q has no dependency on mutable config, state, NAV or the original settler. A delivery can consume 1..16 supported claims at this same Q plus key funding. Every claim's ClaimSpend redeemer must contain the **same semantic delivery-entry list**. The list identifies every consumed Q claim exactly once and allocates distinct receiver outputs. It may include claims from different vaults at this common guard, but no other script-input family, mint, withdrawal, certificate, vote, proposal or treasury action.

For each consumed claim c and its allocated output o:

$$\text{DestinationMatches}(o, c) \land \forall a,\ V(o)[a] \geq V(c)[a]. \tag{10}$$

Matching includes full supported credentials, exact NoDatum/InlineDatum semantics, and no attached reference script. Preserve every actual asset, including surplus or externally funded value, not merely the minimum declared in the datum. Fees and submitter change cannot consume that property. Any added output value is externally funded. The public candidate has no delivery reward.

7.3 Delivery changes location, not economics

Delivery does not consume the accounting State, read current NAV, convert quantities, or mint/burn shares. Share claims already participate in P&L; asset claims already represent a fixed underlying quantity outside remaining share backing. Anyone can fund fixed delivery without redirecting it, subject to real transaction feasibility and network access.

A third party can voluntarily create a funded look-alike at Q. The guard can preserve its actual property without making it an authentic vault claim. Indexers authenticate genuine settlement lineage before attributing it to the vault. Q's preservation guard grants no rights to mint shares or debit another object.

The local wire-format checks cover candidate shape and selected preservation/allocation primitives. The full multi-handler rule that all claims use the same list, compiled receiving compatibility, and ledger-resource feasibility have not been demonstrated. Source and test authors are not independent. [6], [13]

7.4 Direct-at-settlement delivery: a bounded semantic variant

A native implementation MAY support `delivery_at_settlement` through an authenticated Section 2.5 mapping. This is a semantic variant, **not** CTVS-WIRE-2 Direct Custody Profile, not an assigned new on-chain profile integer and not a claim of literal ERC-7540 compatibility. No deployed implementation, compiled mapping or new reference-wire encoding is supplied here. [1]

The initial variant is deliberately narrow: full-fill requests, one authenticated snapshot per batch, fixed authorized request fees, the same participation cutover as Section 3, one final protected allocation per request, and the existing carry-to-receiver storage policy. Do not mix funded-claim and direct-delivery outcomes inside one batch in this initial variant. Epoch, partial-fill and in-flight semantics are not inherited.

For request i, derive the gross economic quote and all state/supply/fee deltas from Section 5 or the mapping's explicitly supported equivalent. Let q_i be deposit shares or w_i redemption assets and $c_i = R_i + B_i - e_i + t_i$, where t_i is externally funded output top-up. Create a final output to the exact committed receiver/datum with the economic asset/quantity and the entire c_i allocation. For ADA redemption, the required lovelace is $w_i + c_i$, not the maximum of the two. Native-token economic quantities are exact; all additional ADA has an explicit source and owner.

Every consumed request must be covered once. Final outputs, accounting continuations and fee outputs are disjoint under the initial allocation rule; the native transaction family must additionally prove that no external script reuses the same value as another protected payment. Complete identity/role authentication, actually executed authority, bounds, expiry, cap/liquidity, terms binding, gross share accounting and exact mint/inventory changes remain required.

Settlement creates no claim and performs no later delivery action. A wallet records one `settled_delivered` outcome with the actual final output references; it must not also credit an issued-but-unclaimed share balance or a fixed asset claim. Before consumption, the request retains its declared fixed cancellation/expiry rights. Once settled, those rights are spent and the user holds the final output. If any required predicate fails, the economic transition does not partially succeed; real ledger collateral handling remains separate.

The native request or immutable mapping-bound terms must commit this disposition before execution. Enabling it on the existing Direct Custody Profile body, whose fixed meaning is claim creation, is forbidden. A future reference

implementation would need an explicit new profile/wire design, fixtures and migration assessment; this paper assigns none.

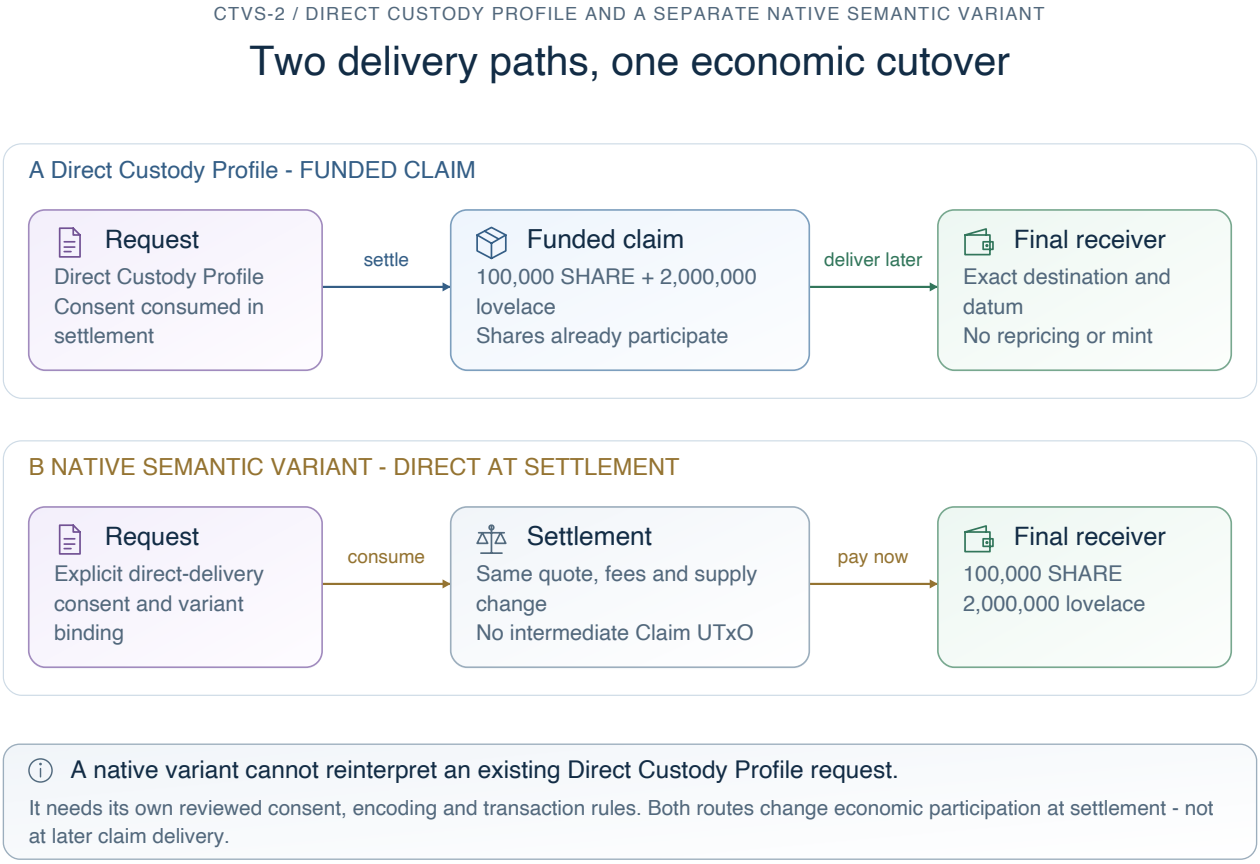


Figure 5: *Two settlement dispositions, one economic cutover. The Direct Custody Profile creates a funded Claim for later delivery; the native direct-delivery variant pays the final receiver inside settlement. One request interpretation cannot silently switch between them.*

7.5 Direct-delivery example and evidence gate

Use Section 9's three-request mixed settlement and unchanged snapshot quotes. Replace only the three funded-claim outputs with final outputs at their respective committed destinations: A receives 100,000 SHARE plus 2,000,000 lovelace; B receives 50,000 SHARE plus 2,000,000; C receives 50,000 UNIT plus 2,000,000. Successor state, +99,500 net SHARE mint, 3,000,000 execution reward, 5,400,000 external-funder change and 600,000 network fee remain the same. All input/output asset totals are unchanged. There is no Claim datum, ClaimSpend action or second delivery fee.

This is a value-conservation example of the proposed native variant, **not a valid Direct Custody Profile transaction** and not an implemented native builder. Its acceptance gate requires native encoding/consent, real guard execution, compiled same-destination/reused-output negative fixtures, actual ledger acceptance, measured output costs and independent indexing that records no ghost claim. The recorded checks cover its arithmetic only. [12]

8. Cancellation, deadlines, and pause behavior

8.1 Recoverable envelope, independent of settlement economics

The fixed recovery guarantees below remain mandatory for the Direct Custody Profile and the claimed refundable-escrow semantics. Native mappings must demonstrate equivalent protection of each user's authenticated attributable property; they cannot infer one user's entitlement to an entire shared input. A mere adapter cannot add refund rights a native guard does not enforce.

A **settlement-eligible** Request must pass the full body, funding, terms, fees, pricing, amount and mode checks. A **recoverable** Request requires only supported outer version and Recovery structure with valid fixed refund, controller and expiry. No active batcher, open pause flags, valid settlement budget, current quote or accepted terms binding is required for fixed refund.

The cancellation recipe consumes one Request plus key-controlled external funding, and no State, other script family, mint, withdrawals, certificates, governance or treasury actions. Resolve its own input and reject protected STATE/ID misuse before interpreting Recovery. Before expiry, ControllerCancel requires the committed key; afterward it remains available. ExpiryRefund permits anyone when the whole validity range lies at/after the deadline.

The fixed refund output must match the exact destination/datum, have no reference script, and satisfy $V(o_r) \succeq V(r)$ over every actual asset. This returns storage, the entire unearned budget, surplus ADA and unexpected native assets without a settlement charge or accounting mutation. All fees/top-ups are external.

The handler MUST branch through raw Data to the stable envelope before economic downcasting. The `economic_body: Data` boundary is the concrete candidate mechanism, not a Python-field shortcut. An unsupported body, zero amount or invalid budget may remain refundable; an unknown outer version, undecodable Recovery, invalid refund destination or unsupported Controller has no universal rescue promise. Extremely large actual value or destination data may still prevent construction. Full-value preservation is safety, not unconditional liveness. [6], [13]

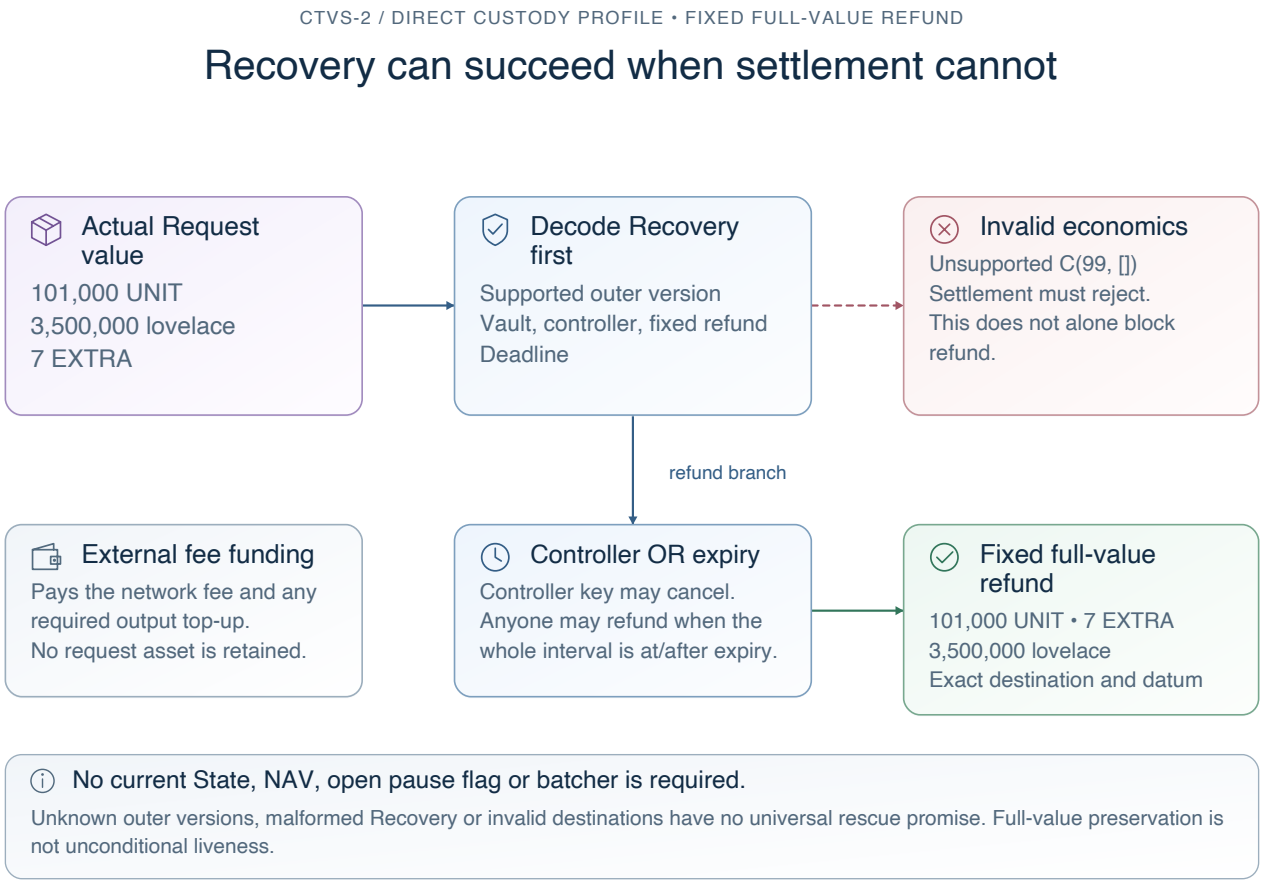


Figure 6: *Recovery is independent of settlement eligibility. A supported Recovery envelope, authority or expiry condition, and exact full-value refund can remain valid when the economic body cannot settle. External funding pays the refund transaction fee.*

8.2 Exact interval semantics and conflicts

Normalize the supported transaction validity interval to nonempty $I_{tx} = [\ell, u)$ with finite bounds and $\ell < u$, using the actual pinned ledger/library semantics. For a deadline d:

$$\text{SettleEligible} \Rightarrow u \leq d, \qquad \text{ExpiryRefundEligible} \Rightarrow \ell \geq d. \tag{11}$$

Equivalently, every permitted settlement time is before d and every expiry-refund time is at/after d. A boundary-straddling interval is not sufficient; neither is a builder's wall clock. Any enabled valuation window must contain the complete interval. Required slot/POSIX conversion and open/closed-bound correspondence remain actual-ledger evidence gates.

Controller cancellation and settlement can compete before expiry, but both spend the same request reference. At most one survives on a given accepted history. An off-chain selection notice does not reserve the input or change the datum. An indexer must reverse orphaned claims/settlements on rollback. [13], [6]

8.3 Pause matrix

Action	Entry pause	Exit pause	Requires settler
Settle deposit	Blocked	Unaffected	According to membership policy
Settle redemption	Unaffected	Blocked	According to membership policy
Create candidate request	May still create an output; acceptance not promised	Same	No
Controller cancellation	Unaffected	Unaffected	No
Fixed expiry refund	Unaffected	Unaffected	No
Funded-claim delivery	Unaffected	Unaffected	No

Request creation cannot be advertised as admission merely because it succeeds while settlement is paused. Providers should warn users and expose the likely execution status. Pausing a vault does not confiscate existing pending escrow or funded claims.

9. Worked UTxO transactions and datum representations

9.1 Conventions and reachable reference basis

UNIT is a hypothetical native underlying, distinct from the authenticated vault policy p. All amounts are integer base units. The numbers for storage, fees and collateral funding are **arithmetic assumptions**, not measured network minima. References contribute no spendable value. All protected outputs have inline protocol data where required and no reference script.

Use $V_a = V_s = 1$, entry/exit markup 100 basis points, old sequence 7, $A_0 = S_0 = 1,000,000$, $F_0 = 0$, and state reserve 4,000,000 lovelace. This mature one-to-one state is reachable through zero-yield user flows and needs no managed State field. Terms enables both asynchronous directions. Each request reserves 2,000,000 lovelace plus a 1,000,000 budget/fee. Let `hK` be the correctly committed terms hash, `D` a complete receiver Destination, and `Frefund` its fixed refund Destination.

9.2 Example A - independent deposit request

A user supplies 101,000 UNIT and 6,000,000 lovelace. Its outputs are the pending Request with 101,000 UNIT and 3,000,000 lovelace, plus 2,600,000 lovelace of change; the illustrative network fee is 400,000. No State is consumed and no SHARE is minted. Use a future positive deadline d and key k:

```
Recovery = C(0,[p,C(0,[k]),Frefund,d])
RequestBody = C(0,[hK,C(0,[]),101000,99900,
                   D,2000000,1000000,1000000])
Request = C(1,[CTVS,2,Recovery,RequestBody])
```

The originating reference `ra#0` is learned after creation; it is not stored in Request. Another deposit `rb#0` independently offers 50,500 UNIT with a minimum of 49,900 shares and the same budget convention. All CTVS economic totals remain unchanged until settlement.

Illustrative reader interpretation. Accepted request `ra#0` escrows 101,000 UNIT without realizing SHARE or changing participating supply. A quote may estimate 100,000 SHARE, but realized output stays unknown until settlement. Example C's settlement against the authenticated old State creates a funded 100,000 SHARE Claim. Delivery changes ownership and claim status without repricing or minting again. If settlement is orphaned, the claimable result rolls back; the request is pending only if it survives on the replacement chain. The pinned reader source implements one interpretation of CTVS-1 Section 13, not a live-indexer result.

9.3 Example B - independent redemption request

A holder supplies 100,000 SHARE and 6,000,000 lovelace. It locks 50,500 SHARE and 3,000,000 lovelace in `rc#0`, returns 49,500 SHARE and 2,600,000 lovelace as change, and pays the illustrative 400,000 fee. The body uses kind `C(1,[])`, offered 50,500 and minimum output 49,900 UNIT. All 100,000 shares remain participating supply; no burn occurs at request creation.

9.4 Example C - mixed snapshot settlement

All three requests use the same old State and fixed per-request markup:

Request	Offered property	Asset fee	Net admission / gross debit	Funded economic claim
ra deposit	101,000 UNIT	1,000	+100,000 backing	100,000 SHARE
rb deposit	50,500 UNIT	500	+50,000 backing	50,000 SHARE
rc redeem	50,500 SHARE	500	-50,500 backing	50,000 UNIT

The successor is $A_1 = S_1 = 1,099,500$, $F_1 = 2,000$, sequence 8. Physical UNIT is 1,101,500, exactly backing plus fees. Net SHARE mint is +99,500. An eligible key settler contributes 6,000,000 lovelace; the fee is illustratively 600,000.

Transaction object	UNIT	SHARE	Lovelace
Spend State + STATE(1)	1,000,000	0	4,000,000
Spend ra	101,000	0	3,000,000
Spend rb	50,500	0	3,000,000
Spend rc	0	50,500	3,000,000
Spend settler funding	0	0	6,000,000
Mint SHARE	0	+99,500	0
Output 0: successor + STATE(1)	1,101,500	0	4,000,000
Output 1: claim A	0	100,000	2,000,000
Output 2: claim B	0	50,000	2,000,000
Output 3: claim C	50,000	0	2,000,000
Output 4: exact reward	0	0	3,000,000
Output 5: settler change	0	0	5,400,000
Network fee	0	0	600,000

UNIT: $1,151,500 = 1,101,500 + 50,000$. SHARE: $50,500 + 99,500 = 150,000$. ADA: $19,000,000 = 4,000,000 + 6,000,000 + 3,000,000 + 5,400,000 + 600,000$. State/config reference/value checks are additional to these equalities.

9.5 Example C - exact two-way redeemers

Let entries be sorted by their actual transaction-ID bytes, with `ra#0 < rb#0 < rc#0` solely as an illustration. Let s0 be the consumed State reference, kB the required reward key, and SHARE the full native Asset constructor:

```
entries = [C(0,[ra#0,1,0]), C(0,[rb#0,2,0]), C(0,[rc#0,3,0])]
StateRedeemer = C(0,[CTVS,2,
    C(4,[0,entries,kB,C(0,[4])])])
# Every Request input has the following purpose-bound redeemer:
RequestRedeemer = C(1,[CTVS,2,C(0,[s0])])
MintRedeemer = C(3,[CTVS,2,C(1,[s0])])
new_state = C(0,[CTVS,2,p,hK,8,
                 1099500,1099500,2000,4000000,0])
claimA = C(2,[CTVS,2,p,ra#0,s0,hK,
              SHARE,100000,D_A,2000000])
```

Replace all symbolic references/hashes/destinations with their exact nested encodings before constructing a transaction. The entry carries no quoted amount: all amounts are recomputed. `claim_topup=0` means no additional ADA is assigned beyond the request remainder. Output 4 is Some reward index; zero summed fees would require None. Request acknowledgments cannot point merely to a reference input or to a State pause/top-up action.

9.6 Example D - independent claim delivery

Claim A already holds 100,000 SHARE and 2,000,000 lovelace. A new submitter provides 4,000,000 lovelace. Output 0 pays the exact receiver/datum all 100,000 SHARE and 2,000,000 lovelace; change is 3,600,000 and fee 400,000. ClaimSpend is `C(2,[CTVS,2,C(0,[[C(0,[claim_ref,0])]])])`. No State or config is consumed, no quote is recomputed, and no shares are minted/burned.

For claim C the delivered asset quantity is fixed at 50,000 UNIT, independent of later NAV. Multi-claim delivery uses one identical semantic list in each ClaimSpend, complete coverage and distinct outputs. A receiving script is not proven spendable merely by constructing its output.

9.7 Example E - cancellation and exact expiry refund

Instead of settling ra, consume it into its immutable refund with all 101,000 UNIT and 3,000,000 lovelace. With 4,000,000 external lovelace, change is 3,600,000 and fee 400,000. Controller cancellation uses RequestSpend inner `C(1,[refund_output])` and the required key. After expiry use inner `C(2,[refund_output])` and an interval wholly at/after d. No settlement reward is charged; no State is required.

9.8 Example F - unsupported economics, supported recovery

Retain ra's valid Recovery but replace `economic_body` with `C(99,[])`. Suppose the actual value is 101,000 UNIT, 3,500,000 lovelace and seven EXTRA tokens. Settlement fails. Recovery must not first interpret that body as RequestBody: a valid controller cancellation or expiry refund returns all actual value to Frefund. With 4,000,000 external lovelace and fee 400,000, change is 3,600,000. No extra token is silently discarded. An unknown outer version or invalid refund destination does not receive this promise.

9.9 Example G - preservation includes excess claim value

A voluntarily funded supported claim-shaped output declares 100,000 SHARE and 2,000,000 carried ADA but actually holds 2,500,000 lovelace. Fixed delivery passes every share and all 2,500,000 lovelace to its destination. The submitter cannot retain the extra 500,000. With 4,000,000 external funding and 400,000 fee, change is 3,600,000, not 4,100,000. This does not establish authentic settlement lineage or imply an existing UTxO can be enlarged in place; legitimate settlement top-ups must agree with the created Claim.

9.10 Example H - zero net mint, nonzero gross exchange

At $A = S = 1,000$ with virtual balances 1:1 and zero fees, deposit 10 assets and redeem 10 pre-existing shares in one snapshot batch. State backing and supply are unchanged; net SHARE mint is zero. The redemption shares can fund the depositor's claim while deposit assets fund the redeemer's claim. State/Request handlers still enforce both gross obligations; no MintPurpose invocation is assumed for zero mint.

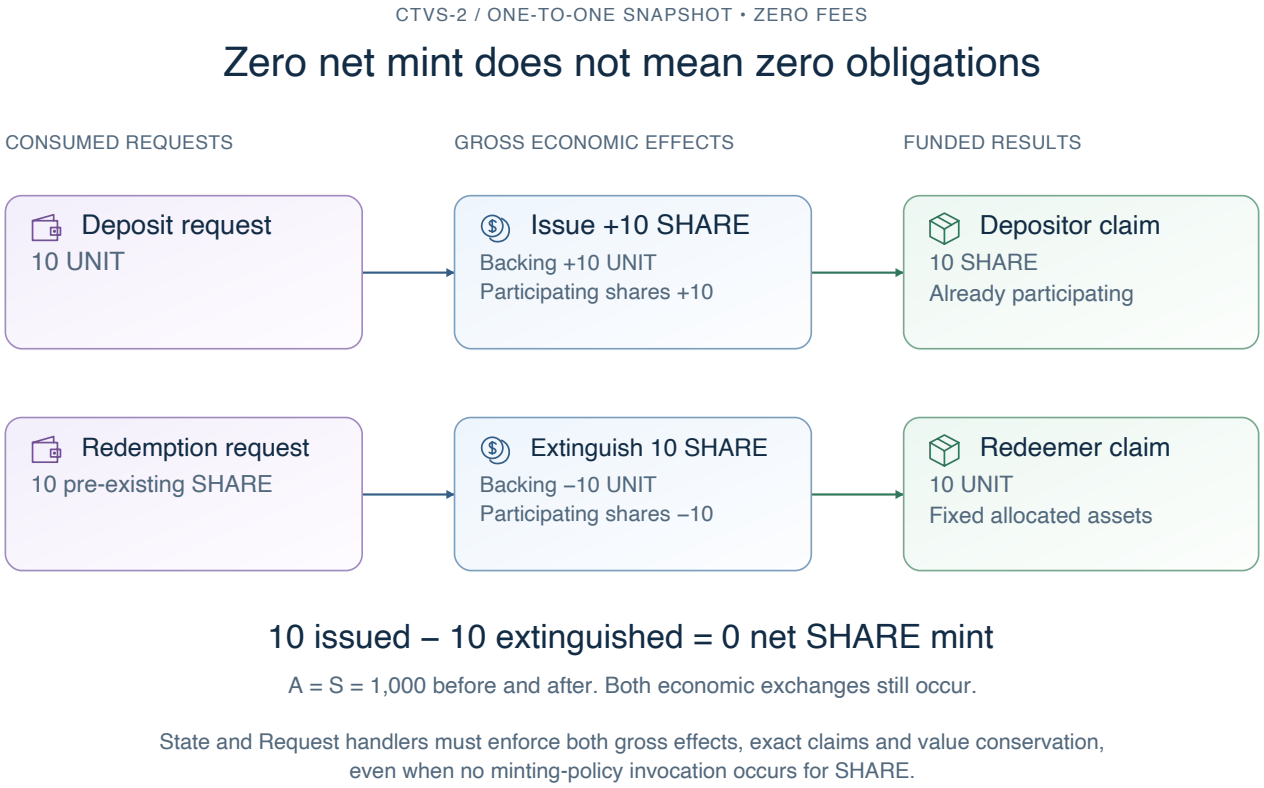


Figure 7: *Zero net mint still contains two gross economic exchanges. The ledger mint field is zero, but State and Request handlers must validate ten shares issued, ten extinguished, and both funded results.*

9.11 Managed-price illustration outside the Direct Custody Profile

Under a separately specified managed state, a pending 100,000-share redemption at indicative A=S=1,000,000 may later face accepted backing 850,000 with unchanged supply. With $V_a = V_s = 1$ and zero fees, the gross quote is 85,000 assets. A minimum of 90,000 blocks settlement; 84,000 permits the economic quote subject to liquidity and all guards. Once funded, an 85,000-asset claim is no longer repriced. The Direct Custody Profile cannot encode that discretionary NAV update; Section 10 defines the additional evidence required.

10. Managed NAV, protocol adapters, and in-flight exposure

10.1 The simple admission boundary

This is a concrete integration boundary, not a universal strategy workflow. One actual invested-vault mapping is prioritized for acceptance; lending/ALM implementations do not become conformant through a generic `managed` capability name.

The conservative integration path admits a deposit and issues shares before the strategy deploys capital. It realizes sufficient liquidity before settling a redemption. This keeps refundable requests and funded claims outside the strategy's internal state machine.

A permissioned DEX or lending adapter may then operate on assets already owned by the vault under the declared manager rules. The adapter specifies allowed scripts, position identity, output data, valuation evidence, liquidity constraints, and permitted authority. The standard does not force the downstream protocol to accept an unauthorized operator.

10.2 Exact state-basis binding and report acceptance

A managed report must identify vault, terms, position/liability perimeter, accepted report sequence, valuation time/window, backing and liquidity under a separate exact managed state. The conservative absolute-NAV candidate additionally binds the **exact consumed accounting-state reference** it values. A report for s0 at 100 must not overwrite s1 at 110 after a ten-asset deposit. Reject/revalue or implement a precisely specified and verified cash-flow reconciliation algorithm; an unnamed bridge is not a solved mechanism.

The accepted report head must be authenticated; an old signed unspent output is not proof of the latest accepted report. The whole transaction interval must fit report validity and request deadline. Exact state binding has a cost because concurrent operations can stale reports; benchmark it. Fee crystallization occurs once in its declared order before user pricing.

The Direct Custody Profile has no NAV field or managed update action. A Merkle root or signature proves neither complete ownership nor fair valuation without its surrounding rules. Permission to settle does not confer permission to invent NAV, and reported positive backing is not liquid assets. No managed-protocol or trusted-NAV test is included in the local wire-format results. [6], [13]

A `Price` conversion of an owned downstream position is an input to managed valuation, not an absolute report overwriting the vault's backing. Native denominations and rate scales follow CTVS-1 Section 6.5; authenticated report schemas must bind any scale used to interpret a signed value. The profile must reconcile positions, debt, fees and cash flows before pricing requests. It must independently establish liquidity and authorized execution; a correct conversion assertion supplies neither. Exact provider encoding is specified once in CTVS-1 Appendix B.

10.3 In-flight exposure is excluded until separately specified

If pending assets are committed downstream before final pricing or recovery ceases to be freely executable, the position is no longer `PendingEscrow` under this specification. It requires a distinct, explicit ownership/recovery model before any execution-conformance claim. That workflow requires a separate specification. A timeout cannot reverse an investment, create liquidity or bypass a downstream operator's permissions. Retain the actual state and rights in the read view; do not label the position fully refundable or funded merely to fit a common enum.

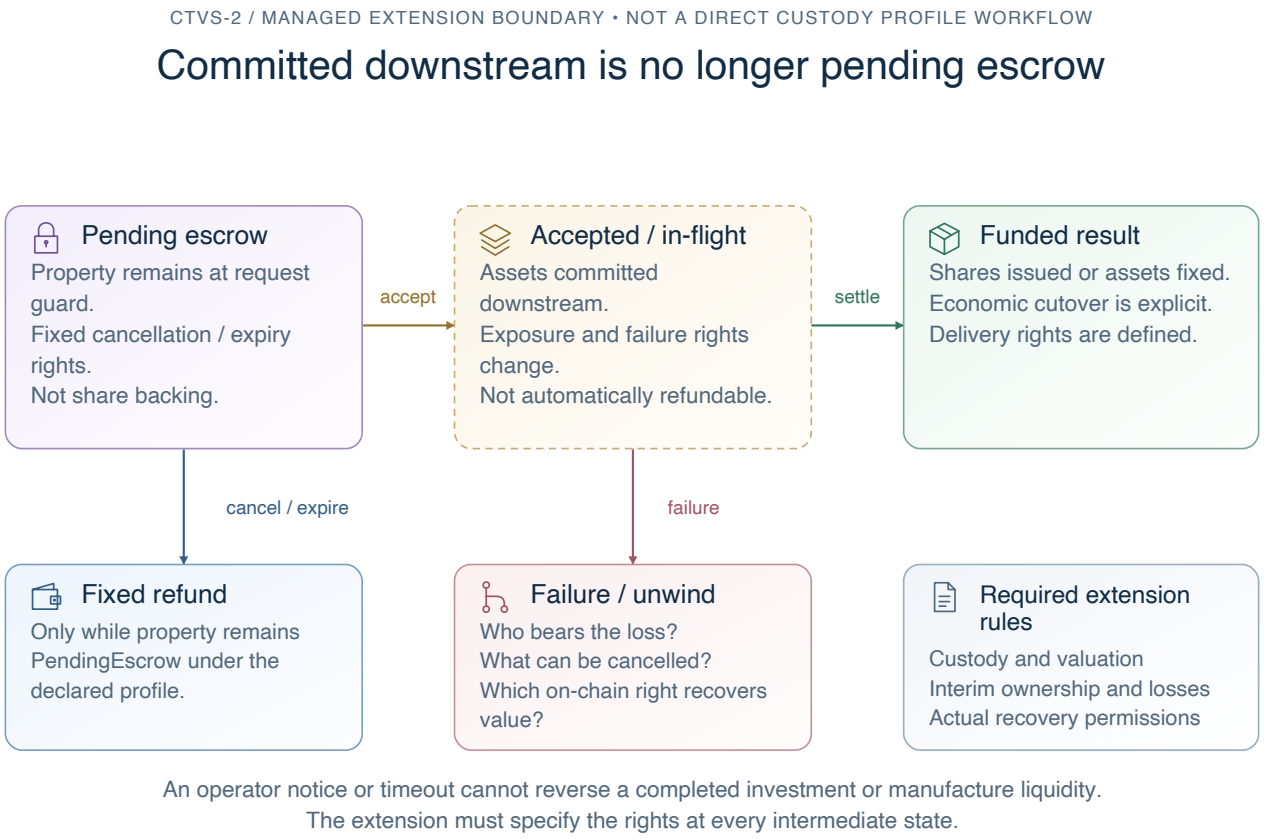


Figure 8: *Deploying pending assets creates a different ownership state. Accepted in-flight assets cannot continue to be described as fully refundable escrow; the extension must define loss, cancellation, unwind, valuation, and recovery rights.*

10.4 Leverage and multi-step positions

A looping strategy may need several distinct transactions to borrow, exchange, and redeposit. CTVS does not promise an atomic loop or an exact completion time. The manager's intermediate custody, debt, and risk limits must be accounted for in the chosen managed profile.

The common vault interface exposes the resulting share exposure and availability; it does not erase downstream liquidation, oracle, permission, or settlement risk. A standardized share asset is not automatically suitable collateral for another lender.

10.5 Managed acceptance cannot be inferred from a direct trace

Before any managed-profile conformance claim, implement one real permissioned lifecycle against a pinned target protocol: deposit admission, allocation, authenticated position custody, debt and fee recognition, accepted valuation, liquidity realization, redemption and stage-specific recovery. Script-controlled user authorization requires its own real guard and successor tests. Downstream operators need not become permissionless; evidence must establish the permissions that actually exist. [13], [6]

11. Deferred features and bounded compatibility

11.1 Features outside the initial execution variants

Feature	Current decision	Requirement before enabling it
Multi-batch epoch pricing	Deferred	Authenticated rate/eligibility, capacity consumption, participation timing and residual treatment
Partial fills	Deferred	Original request identity, cumulative bounds/fees, remaining ownership and cancellation
Pre-pricing investment or unfunded withdrawal tickets	Deferred	Actual interim exposure, liability and recovery state machine
Arbitrary routes, claim-and-redeposit and cross-vault settlement	Deferred	Executed complete authorization/allocation across the actual script family
Transferable claim rights	Deferred	Explicit ownership transfer, replay protection and funded delivery semantics

No sample algorithm in this specification is a substitute for those specifications. The eight-field reference RequestBody contains no optional epoch, partial-fill, bearer-right or generic capability fields. Its opaque economic-body boundary exists to preserve fixed recovery, not to authorize arbitrary module interpretation.

11.2 Inventory accounting is a common mapping obligation

A native inventory implementation must derive $S = N - I_s$ from authenticated economic issuance and extinction. Pending-redemption shares and issued claim shares participate; they are not unissued inventory. The common gross obligation checks still apply even when no native mint occurs. The internal inventory arrangement and replenishment are implementation-specific, but their conservation and authorization require a concrete mapping and evidence. The Direct Custody Profile remains mint/burn. See CTVS-1 Section 5.3.

11.3 Direct delivery is not generalized routing

The narrow `delivery_at_settlement` variant is specified in Section 7.4 rather than grouped with an open-ended routing framework. It replaces an intermediate claim with the exact final output only after preserving complete accounting, authorization and attribution. It neither relaxes the Direct Custody Profile transaction family nor enables spending the receiving script in the same transaction without a separately proved composition.

11.4 Existing terms and user property remain protected

Native governance/update mechanisms need not copy an immutable configuration lock. They must authenticate who can update what and bind each pending request to the applicable terms or a precisely bounded authorized update rule. No silent increase of fees, receiver substitution, asset substitution or deterioration of a committed recovery right is acceptable. Funded claims remain allocated property and cannot be repriced by migration. The implementation must expose operator/manager changes affecting availability. Direct Custody Profile's immutable terms and no-close policy remain unchanged; no generic governance framework is added.

12. Wire schema and infrastructure interface

12.1 One shared primitive specification; exact async records

CTVS-1 Section 10.1 is the authoritative definition of `Credential`, `Option`, supported `Address`, `Destination`, `Asset`, `OutRef`, key-only `Controller` and `Settlers`, including bounds and datum variants. It is not duplicated here. `C(t,[...])` means Plutus constructor Data, not a JSON object or CBOR major type. The reference magic is bytes CTVS; its wire integer remains 2 and profile remains 0.

The following exact Request/Recovery/Claim records remain normative **for that reference encoding**. The native conformance route in CTVS-1 Section 2.5 requires a separately versioned, authenticated mapping of another implementation's bytes. It does not permit reinterpretation of these records or Direct Custody Profile's claim-only disposition. The shared candidate files define the exact records and commitment domain. [7], [12]

12.2 Exact Recovery, Request and RequestBody

```
Recovery = C(0,[
  vault_policy, controller, refund_destination, deadline_posix_ms
])

Request = C(1,[
  magic, wire_version, recovery, economic_body
])

RequestBody = C(0,[
  terms_hash, kind, offered, minimum_output, receiver,
  storage_lovelace, execution_budget, settler_fee
])

kind = C(0,[])                              -- exact-gross deposit
     | C(1,[])                              -- exact-share redemption
```

The Request has exactly four fields. Recovery has four fields, and the supported economic body has eight. The body's type in the outer envelope is **Data**, not an eagerly validated `RequestBody`. Full-fill behavior, snapshot pricing, funded-claim delivery, and carry-to-receiver storage policy are fixed by the Direct Custody Profile rather than represented by unconstrained strings.

The recovery branch validates the outer version and recovery structure, then the required authority/time condition and entire-value refund. It does **not** first validate economic kind, positive amount, success receiver, fee budget, terms equality, or current state. Thus an economic body with an unsupported constructor can fail settlement while remaining refundable under a supported envelope.

Do not implement this by fully deserializing a typed economic request before dispatching on `Cancel`. In an Aiken handler, accept the appropriate raw Data boundary and perform the recovery projection before economic downcasting. The uncompiled type declaration uses `economic_body: Data` explicitly. It still requires the compiler/entry-point gate. [9]

For settlement, additionally require the expected vault and terms; supported kind/mode; positive offered and minimum output; exact value funding; `settler_fee <= execution_budget`; and full fulfillment. Native deposit value is d units of u plus R_user+B_user ADA. ADA deposit value is d+R_user+B_user lovelace. Redemption value is r shares plus R_user+B_user ADA. No other native assets enter this closed settlement shape.

The request's own originating transaction hash does not appear in its datum. Its identity is the created output reference, learned after transaction construction. The controller need not be the funding owner or success receiver. A candidate request naming somebody does not prove they initiated it.

12.3 Exact Claim and creation semantics

```
Claim = C(2,[
  magic, wire_version, vault_policy,
  request_ref, state_ref, terms_hash,
  asset, economic_quantity, receiver, carried_lovelace
])
```

Exactly ten fields are required. `request_ref` and `state_ref` refer to consumed predecessors. Neither is the creating claim transaction ID. The actual creating transaction and output index are external identity, preventing a self-reference in construction.

At settlement, the asset and quantity are derived from the request and old state. Carried ADA is:

$$R_{\text{user}} + B_{\text{user}} - e_{\text{user}} + t_{\text{external}}. \tag{12}$$

An ADA claim holds economic quantity plus carried ADA; a token claim holds the exact token quantity plus carried ADA. Legitimate creation has no unexplained surplus. Delivery preserves **all actual input value**, including voluntary top-ups in look-alike outputs, not only the declared quantity.

The shared claim validator requires no vault state, config, current NAV, or settler availability. Authentic settlement lineage is an indexer obligation unless a separately specified beacon provides local authentication. A claim-looking output does not prove a vault issued a receipt; it only exposes the guard governing the property actually locked there.

12.4 Exact execution envelopes and deterministic lists

```
StateRedeemer   = C(0,[magic,wire_version,StateAction])
RequestRedeemer = C(1,[magic,wire_version,RequestAction])
ClaimRedeemer   = C(2,[magic,wire_version,ClaimAction])
MintRedeemer    = C(3,[magic,wire_version,MintAction])

Batch = C(4,[state_output,entries,reward_key,Option<output_index>])
SettleEntry = C(0,[OutRef,claim_output,claim_topup])
RequestAction = C(0,[state_ref])      -- acknowledge Batch state spend
              | C(1,[refund_output]) -- controller cancellation
              | C(2,[refund_output]) -- expiry refund
ClaimAction = C(0,[entries])
DeliverEntry = C(0,[claim_ref,receiver_output])
MintAction = C(0,[config_output,state_output]) -- genesis
           | C(1,[state_ref])               -- nonzero supply update
```

State's direct, fee, pause and reserve action tags/fields are defined in CTVS-1 Section 10.4. Lists contain 1..16 entries, with settlement further bounded by Terms.max_batch; out-references are unique and lexicographically ordered. Indices are bounded integers checked against actual transaction arrays. Every consumed Q claim uses the same semantic delivery list; every consumed P request uses an acknowledgment to the selected authentic state. All

amounts are recomputed; entry.topup is external carried ADA only. The Direct Custody Profile adds no optional capability continuation or authoritative quoted totals.

12.5 Shared encoding rules and recovery boundary

CTVS-1 Sections 10.5–10.7 are authoritative for semantic Data equality, original versus reserialized bytes, the fixed commitment domain, recursive opaque-data limits, candidate size bounds and the remaining builtin/independent-decoder gate. They apply unchanged to the exact records here. Lists, references, amounts and their aggregates must satisfy both schema bounds and the actual transaction context.

Recovery deliberately does not first validate the complete economic body. Unknown outer versions and malformed refund/controller envelopes retain no implicit rescue path. The preserved guards reject attached reference scripts and unsupported datum/address forms under the Direct Custody Profile; a native mapping must explicitly specify its own accepted forms and exact destination semantics. Source-level publication cleanup must not change signed transaction bytes or on-chain commitments. [7], [12]

12.6 Common request/claim response contract

Use CTVS-1 Section 13's common envelope and value-state rules, not a second transport or provider standard. For every request, the async response additionally identifies original request ID, observed source reference, native/reference implementation binding, applicable terms, controller and execution authority, economic receiver and refund destination, offered property, user bound, fee/storage partition and deadline.

All offered quantities, bounds and funded entitlements retain their identified asset's ledger units. For native `Price` use, the response also identifies the approved provider, evidence and settlement pricing basis through the shared implementation binding. Human-readable decimal formatting cannot reprice an immutable Claim.

Field group	Required interpretation
Lifecycle	`pending_escrow`, `claimable`, `delivered`, `settled_delivered`, `refunded` or explicit unsupported native state
Participation	Whether offered/issued shares still participate and whether deposit assets have become backing
Recovery	Exact controller/expiry conditions and fixed entitlement; `supported`, `unavailable`, `unknown` and `unsupported` are distinct
Delivery	`funded_claim` or `delivery_at_settlement`, committed before execution; a native mapping cannot rewrite a Direct Custody Profile request
Pricing	Actual settled basis or explicitly indicative future estimate; never a reservation implied by an API response
Realized output	Correct asset and quantity, reserve owner, transaction/output references and authenticated lineage

A `claimable` response requires actual funded property and verified settlement lineage. A pending estimate cannot populate a realized amount. Direct-at-settlement completion records final outputs once and no intermediate claim. Economic input, carried ADA, refundable budget and execution charge retain their distinct meanings, even when the native representation combines fields.

An unknown outer recovery envelope, an unsupported controller, invalid economics with a valid recovery envelope, a paused settlement and unavailable liquidity must be distinguishable. A supported refund is not a guarantee that

an arbitrarily large malformed input fits a transaction. A hosted builder's statement of conformance is evidence to verify, not authority. The lifecycle examples in Section 9 and the common response fields in CTVS-1 Section 13 state the publication contract. The Direct Custody Profile reader and local reader tests give one concrete implementation, not a deployed response service.

12.7 Authentic lineage and rollback

Derive originating request IDs after creation; derive claim IDs from the creating settlement transaction. Authenticate genuine claims by accepted state/request/claim lineage, not a plausible datum alone. A look-alike can lock voluntarily supplied property without representing a vault liability. No per-request NFT is required by this profile.

Snapshots must refer to a single chain point. Keep freely held shares, pending-redemption shares, issued-but-unclaimed shares, pending deposits and fixed asset claims distinct. Reversing a settlement removes its authentic claimable result and restores a request only if present on the replacement history. Reversing delivery alone restores the unspent claim. Mempool observations are provisional. The model's projected rollback does not establish production-indexer replay. Independent construction and indexing remain acceptance gates. [6], [13]

13. Formal properties and proof sketches

13.1 At-most-once transition

A pending request is an ordinary UTxO. Settlement, cancellation, and expiry refund all consume that same output reference. Under ledger no-double-spend validity, at most one such transition can occur on a single accepted history. A claim's delivery similarly consumes its unique output. An indexer must apply this property per chain history and undo orphaned transitions on rollback.

This is not sufficient by itself: each spending branch must still enforce its complete output obligations and disallow unauthorized alternative paths.

13.2 Ownership conservation

This argument applies both to a funded-claim settlement and to the Section 7.4 direct-at-settlement variant. In the latter, the economic result is already at the final receiver; omit the intermediate claim/delivery step and do not count another supply or backing change. The remaining formulas use the Section 5 snapshot algorithm.

For deposits, creation and refund do not change A or S. Settlement admits n_i, accrues f_i, and issues q_i to the committed final destination or funded claim, according to the supported variant. Any later claim delivery transfers q_i without issuance. For redemptions, creation locks r_j without reducing S; settlement removes g_j from backing, accrues h_j, extinguishes r_j, and allocates the fixed w_j to the final destination or funded claim. Any later claim delivery transfers that same fixed w_j.

Consequently, settlement is the sole economic participation boundary. Delaying claim delivery cannot create extra supply or reallocate NAV between the claimant and remaining shareholders.

13.3 Snapshot batch rounding

Let successor adjusted balances be $X_1 = X + \sum n_i - \sum g_j$ and $Y_1 = Y + \sum q_i - \sum r_j$. When the validated successor is feasible and $Y_1 > 0$:

$$YX_1 - XY_1 = \sum_i (n_i Y - q_i X) + \sum_j (r_j X - g_j Y) \geq 0. \tag{13}$$

Each term is nonnegative by the floor definitions. Hence $X_1/Y_1 \geq X/Y$ for this batch under unchanged external valuation. Rounding does not dilute the adjusted price of the remaining pool. This is not a theorem about truthful NAV, future investment returns, or optimal batch selection.

13.4 Fully funded claim independence

This property applies only when a funded claim is created. Direct-at-settlement completion has no later claim to deliver and therefore requires evidence of the actual final payment instead.

If settlement places the exact promised quantity in a claim and its only valid spend transfers that property to its immutable destination, delivery does not require the vault's solvency at that later time. The claim's asset itself may have external risks; this property concerns allocation, not market value or issuer behavior.

The reference delivery script's common input domain and exhaustive injective mapping are essential. Without them, two claim inputs could independently count one payout and violate the claim-level conservation argument.

13.5 Conditional liveness

Under an available network, valid transaction funding, and the advertised controller proof, a still-pending reference request can be cancelled; after expiry, fixed refund requires no controller or batcher authorization. A funded claim can be delivered without either. These are conditional construction properties, not guaranteed inclusion times.

No unconditional settlement liveness follows from an authorized operator set. No immediate exit follows from positive managed NAV. Downstream liquidity, permissions, and oracle availability remain explicit assumptions.

13.6 Recovery noninterference and complete branch coupling

Conditional recovery independence: after supported outer/Recovery decoding, fixed refund eligibility must not depend on interpretation of economic_body. Changing a body to unsupported Data cannot change the immutable refund destination or give custody to a settler. The handler must reach the recovery branch before economic downcasting; this property remains subject to outer-version validity and actual transaction resources.

Conditional batch completeness: exact set equality, unique entries, and purpose-bound acknowledgments in both directions prevent consumed Requests from being silently ignored or treated as cancellation and settlement simultaneously. They do not alone establish prices, funding or mint. All those obligations must hold for the same covered set.

Conditional exclusive delivery: when every Q claim validates the same exhaustive entry list, allocations are disjoint, destinations match and each output preserves its source's full value, one payout cannot satisfy two claims in that domain. This is not a proof for foreign script inputs excluded by the profile. The local boundary primitives test selected parts; a complete multi-handler implementation and refinement proof remain outstanding. [6]

13.7 Direct delivery does not remove an economic obligation

Let the funded-claim transaction allocate request i economic value v_i and carry c_i to its claim output. A permitted direct variant replaces that output with the same protected value at the committed final destination and removes only the intermediate claim datum/guard. Its state, fee and supply deltas are unchanged. Hence value balance is preserved by substitution, but safety additionally requires native authorization of the disposition, recipient/datum verification

and complete non-overlapping allocation. Arithmetic substitution alone proves no validator accepts it. Section 7.5's example exercises this limited value identity, not the missing compiled obligations.

14. Economics, scalability, and risk disclosure

14.1 Costs and budgets

For m requests, an illustrative batcher reward is $\sum_{u=1}^{m} e_u$. Its realized operating margin depends on network fees, external minimum-output top-ups, infrastructure cost, failed submissions, and opportunity cost. The standard neither fixes profitability nor promises a competitive operator market.

Per-request ceilings can make splitting more expensive, and batch selection can favor requests with higher economic value to the operator. A fixed reward avoids an unbounded fee grab but does not prove fairness or availability. The user must see economic asset fees separately from ADA execution budgets and storage.

Cancellation and fixed delivery have zero protocol reward in the reference profile. Their submitter funds the network transaction. This removes a reward-theft surface but does not subsidize user recovery.

14.2 Throughput and resource limits

Requests can be created independently, while settlement still updates shared accounting. Practical batch size is limited by transaction bytes, execution units, output count, datum sizes, authority witnesses, and current ledger parameters. No fixed transactions-per-second or requests-per-batch figure is asserted.

A release must benchmark increasing batch sizes and worst-case destinations. A provider must not assume every request can fit in a batch merely because its economic minimum is satisfied. Oversized required data must be rejected at the schema boundary, not discovered only after a user has irrecoverably locked funds.

14.3 Material risks

Risk	Required protection or disclosure
Authorized operator censors or disappears	Pending refund and independent claim delivery; no invented forced settlement
False or stale NAV	Accepted report sequencing, interval checks, and explicit reporter trust
Same output counted twice	Complete allocation plus a closed/common verification domain
Budget or reserve theft	Separate value partitions and exact success/refund destinations
Claim created without funds	Actual value check before authentic claimable status
Rounding/fragmentation abuse	Per-request full-fill arithmetic; partial fills disabled
Terms changed after commitment	Immutable binding or separately defined protected update rules
Spoofed request or claim	Validate value/shape and authenticate economic lineage
Downstream execution failure	Explicit managed/in-flight state; real recovery assumptions
Chain rollback	Reversible lifecycle indexing at identified chain points

Compliance with the standard describes an interface and its enforced economic properties. It is not a universal safety rating for an investment strategy.

14.4 Actual size, encoding and signatory constraints

The wire candidate defines finite structural limits, including 1..16 entries and bounded nested recipient data; it does not benchmark a practical batch size. The representative Request/Claim fixture sizes of 310/311 bytes are datum sizes only. Complete serialized outputs, witness/reference scripts, quantities, fees, collateral, required keys and validity intervals determine actual feasibility. Top-ups can change encoding length and require iteration. [13], [2]

Do not infer authorization from the presence of a funding witness where the validator checks explicit required signatories. Do not normalize signed transaction CBOR and keep its old ID. A future zero-withdrawal or capability optimization must explicitly revise the execution profile rather than silently bypass current restricted-purpose rules. [6], [8]

15. Validation and release requirements

15.1 Required evidence

This is the concrete reference-profile acceptance matrix. A native implementation must prove the corresponding common obligations through its Section 2 mapping, not reproduce the reference factory or every reference-local mechanism. Native direct delivery must additionally satisfy its own variant requirements; claim-only tests apply to the funded-claim path. Evidence for one construction cannot be transferred by analogy to another.

ID	Requirement	Required tests or proof artifacts
A-01	Independent request submission	Multiple creations without consuming accounting state
A-02	Valid request values and intent	ADA/native/share partitions; malformed request rejection
A-03	Key controllers; separate capability extension	Actual key witness tests; capability guard evidence only for its own profile
A-04	Full-only settlement	No omitted input, duplicate request, or hidden remainder
A-05	One declared pricing snapshot	Mixed batch; wrong checkpoint; per-request fee rounding
A-06	Economic/supply conservation	Gross issuance/redemption and zero-net-mint cases
A-07	Complete output allocation	Same receiver, repeated indices, overlapping reserve obligations
A-08	Fully funded immutable claims	Wrong quantity/asset/datum; no repricing or mint on delivery
A-09	Cancellation and expiry	Exact interval boundaries and cancel/settle races
A-10	Permissioned settlement without custody discretion	Unauthorized operator; authorized but economically invalid transaction
A-11	Independent fixed delivery	Batcher/controller offline; multi-claim output uniqueness
A-12	Coherent NAV and liabilities	Stale unspent report, overlapping holdings, and missing post-report flows

ID	Requirement	Required tests or proof artifacts
A-13	Rollback-aware lifecycle	Settlement rollback; delivery-only rollback; competing branch replay
A-14	Bounded execution	Measured worst-case batch sizes, datums, reserves, fees, and witnesses
A-15	Independent interoperability	Two decoders/builders/indexers agree on the same transaction histories

15.2 Recorded evidence and publication checks

Both papers refer to one shared evidence record in historical validation evidence; the historical results are not new runs and are not additive across the papers. They cover bounded economic/semantic checks and separate candidate-codec/link checks, not a complete compiled validator. Tests and models share an author; two languages do not establish independent review. [13]

The historical publication check covered candidate-wire preservation, arithmetic examples, illustrative response fixtures, source references, and layout. Its saved results and preflight reports are preserved with the prior edition under publication archive. The historical suites do not establish that their illustrative response schema matches the later reference implementation. The linked implementation has separate compiled-validator and signed local transaction evidence in [11]. Node-backed acceptance, independent interoperability, external review, and deployment remain unverified.

The historical requirement register retains its recorded BLOCKED dispositions; this paper does not rewrite those dated results. The later Direct Custody Profile reference supplies compiled validators and signed local examples for direct operations, requests, settlement, recovery, and Claim delivery [11]. It does not establish node-backed lifecycle acceptance, full resource capacity, independent builder or indexer agreement, managed integration, or production security. Treat each implementation claim according to its own pinned source and evidence, not the older register alone.

15.3 Required model-to-validator refinement

Let $\alpha(T)$ project a real transaction and its resolved ledger context into the specified semantics. The soundness target is:

$$\begin{aligned} &\text{LedgerPrerequisites}(T) \\ &\quad \wedge \text{CompiledValidatorsAccept}(T) \\ &\qquad \Longrightarrow \text{CTVSRequirements}(\alpha(T)). \end{aligned} \tag{14}$$

Define every projection: own script purpose, ordinary versus reference inputs, credential nesting, actual datum variants, required signatories, mint, auxiliary era fields, time bounds, source references and output indices. Decoding bytes and passing amounts into the semantic model by hand is not that refinement proof.

Constructive success is also required: each advertised supported operation has a valid, resource-feasible, ledger-accepted transaction with the intended effect. Reject-everything scripts are not conforming. Negative cases must fail for the intended semantic reason; phase-1-invalid witnesses must not mask missing phase-2 payout checks.

15.4 Compiled end-to-end acceptance trace

The trace below validates the specified **reference construction**. A native mapping supplies its own equally explicit deployment, operation and recovery trace under its actual scripts and custody assumptions; it is not required to deploy F_lock or Q_claim.

```
authenticated F_lock / Q_claim / per-vault T build and genesis
 -> direct deposit with exact State/Config/Terms
 -> independent deposit and redemption Request(Recovery,Data)
 -> purpose-bound permissioned Batch, including zero-net mint
 -> funded Claim creation and fixed delivery to key/script destinations
 -> cancellation/expiry with invalid economics and batcher unavailable
 -> fee collection and state-reserve top-up
 -> independent construction, authentic lineage and rollback replay
```

Run the same trace against malformed encodings, equivalent Data encodings, wrong-role datums, substituted config, wrong-branch acknowledgments, omitted/duplicate requests, output reuse, balanced theft, stale references and forbidden auxiliary actions. Confirm the pinned builtin's commitment bytes and actual ledger datum/hash behavior. Record full transaction bytes, outputs, witnesses, fees, collateral/return, minimum ADA and CPU/memory under a named protocol-parameter snapshot.

The baseline recorded in the repository is Aiken v1.1.22, standard library v3.1.0 and Plutus V3. This identifies the intended baseline, not the latest tooling or a successful compilation. Pin the ledger implementation and network parameters as well. [13], [8]

15.5 Profile-specific and independent-integration gates

A managed-profile claim additionally requires a concrete permissioned position lifecycle with exact custody, liabilities, accepted NAV basis, liquidity realization and real recovery permissions. A script-controller claim requires actual guard/purpose/domain and successor tests. Neither follows from direct-custody or key-only evidence.

A second builder must construct from authenticated public data without calling the first builder's construction implementation. Separate decoder expectations and a separate indexer must agree on transactions, exact fields, origin attribution and rollback. Two wrappers around one library or two codecs by the same author are not independent external validation.

The shared evidence register records local evidence without clearing any compiled-validator or ledger gate. The reference wire remains a candidate. Publishing this document does not update GitHub, deploy a contract, or certify production security.

16. References

ERC-4626, ERC-7540 and CIP-57 were rechecked on 11 September 2026 for their interface context; no fresh contract or deployed-protocol audit was performed. New scope, mapping and delivery requirements are CTVS proposals, not claims of SundaeSwap conformance. Proposed state machines, equations, schemas, profiles, and examples are CTVS design content unless explicitly attributed. Protocol examples describe published architecture, not a certification of current deployed scripts.

[1] ERC-7540: Asynchronous ERC-4626 Tokenized Vaults. https://eips.ethereum.org/EIPS/eip-7540

[2] Cardano CIP-55: Protocol Parameters (Babbage Era). https://cips.cardano.org/cip/CIP-0055

[3] SundaeSwap. Published contract architecture. https://github.com/SundaeSwap-finance/sundae-contracts

[4] Minswap. AMM V2 specification. https://github.com/minswap/minswap-dex-v2/blob/main/amm-v2-docs/amm-v2-specs.md

[5] Cardano CIP-89: Distributed DApps and Beacon Tokens. https://cips.cardano.org/cip/CIP-0089

[6] CTVS-WIRE-2 reference encoding. Exact layouts and transaction obligations are specified in Section 12. Supporting artifacts are listed in [7].

[7] CTVS-WIRE-2 candidate artifacts: schema, CDDL, golden bytes, and Aiken types. These candidate type declarations are uncompiled; the candidate bytes have not been certified against the Plutus builtin.

[8] Aiken standard library v3.1.0, cardano/transaction.ak. Recorded blob `12f8ec80fdfe6c9976d63f3bb701f4fd525d3ec3`. https://github.com/aiken-lang/stdlib/blob/v3.1.0/lib/cardano/transaction.ak

[9] Aiken, Validators. https://aiken-lang.org/language-tour/validators

[10] Steer Protocol. CTVS-1 and CTVS-2 reference workspace, pinned at repository commit `b52962987d3d078d3da7ba05ca973fe2926b101e`. Source and local tests, not independent conformance certification.

[11] Steer Protocol. Reference validation record at the pinned commit. Dated local results and remaining acceptance work.

[12] Historical publication, arithmetic, and schema check results and saved check log. These are not compiled-validator or ledger evidence.

[13] CTVS historical validation evidence. Summarizes semantic and economic checks, candidate-codec checks, and implementation assurance requirements with their recorded limitations. Historical requirement dispositions remain BLOCKED; current implementation results are separately documented in [11].

Appendix A. Scope and delivery checklist

Decision from the scope review	Specification
Preserve exact UTxO/datum hardening	Direct Custody Profile reference construction and all candidate-wire files retained
Stop treating one construction as universal	Common guarantees separated; native-format conformance now explicit
Avoid new transcript/module/permission frameworks	None required by this specification
Keep economic supply and user-bound mathematics	Retained; native inventory/fee rules require precise mappings
Support script-controlled protocol users	Concrete acceptance target, not a fictitious key-only success claim
Avoid mandatory intermediate claim for every native flow	Bounded direct-at-settlement semantic variant added outside the Direct Custody Profile
Consider budget simplification	Algebra checked; existing fields preserved, no silent wire change
Trim unfinished features and duplicated evidence	Epoch/partial-fill workflow algorithms deferred; evidence consolidated once
Make invested-vault support substantive	One pinned managed integration is an explicit unresolved acceptance milestone

The publication source bundle includes both Markdown papers, figure sources, unchanged candidate-wire artifacts, the shared historical evidence record, and PDF build tooling. `python tools/build_whitepapers.py` typesets the papers. The archived publication checker records historical checks; current reference implementation code and dated local validation are linked in [10] and [11]. Final publication QA must inspect rendered equations, tables, headings, cross-references, and links.